

GPSmap 导航定位 App 开发文档

基于 Android 平台的导航定位手机 APP 改进与 GNSS 功能扩展

一、任务

基于 Android 或 iOS 平台或鸿蒙平台，在已有代码基础上改进导航定位手机 APP，实现 2 项新功能，可从坐标转换、GNSS 定位精度评价、GNSS 定位质量评估（可见卫星数、信噪比、DOP 等）、GNSS 导航误差源分析、GNSS+惯性传感器组合导航、GNSS 欺骗与异常检测、基于 CORS 或 PPP 提升定位精度共 7 项中任选 2 项。

参考资料包括：自制导航 App（包含地图、定位、自定义 marker、路线制定、模拟导航等功能）、移动开发实验之百度地图定位 App 的开发、RtkGps 开源项目、导航电子地图应用开发中间件接口规范（GB/T 39584-2020）。

本项目在原 GPSmap 软件已有地图、定位、搜索、路线规划、电子围栏、轨迹记录和拍照定位功能基础上，进一步扩展为六个与 GNSS 导航定位相关的功能模块：坐标转换、GNSS 定位质量评估、GNSS 定位精度评价、GNSS 导航误差源分析、GNSS 欺骗与异常检测初筛、GNSS+惯性传感器组合导航展示。虽然任务要求至少实现两项功能，但本次开发采用模块化方式扩展多项功能，便于从定位结果、卫星状态、误差来源和传感器一致性等角度综合展示导航定位 App 的改进效果。

任务要求选项	本项目实现情况	对应代码位置
坐标转换	实现 GCJ-02、WGS84、BD-09 三套坐标联动显示	updateCoordinateConversionText、gcj02ToWgs84、gcj02ToBd09
GNSS 定位精度评价	实现水平精度等级、速度、航向、定位方式和可用性建议	updateAccuracyEvalText、evaluateAccuracyLevel
GNSS 定位质量评估	实现卫星数、C/N0、星座统计、DOP 估算和质量等级	startGnssStatusMonitor、updateGnssStatus、estimateDop
GNSS 导航误差源分析	实现遮挡、多路径、几何构型差、网络/融合定位等误差判断	updateErrorAnalysisText
GNSS+惯性传感器组合导航	实现手机传感器航向与 GNSS 航向一致性分析	initSensorFusion、onSensorChanged、updateSensorFusionText
GNSS 欺骗与异常检测	实现位置跳变、速度矛盾、卫星状态与精度矛盾等初筛	updateAnomalyDetection、updateAnomalyDetectText

二、算法

2.1 总体路线

本软件采用“地图 SDK 定位结果+Android 系统 GNSS 状态+手机惯性/磁传感器”的综合技术路线。高德地图 SDK 负责地图显示、定位、POI 搜索和路线规划；Android 系统 GnssStatus 接口负责读取真机可见卫星数、参与定位卫星数、载噪比 C/N0、卫星高度角、方位角和星座类型；SensorManager 负责读取加速度计和磁力计，用于估计手机姿态航向。界面层采用“顶部摘要卡片+可展开功能模块”的方式，只在默认界面显示关键结论，用户点击展开后再查看每项功能的详细计算结果。

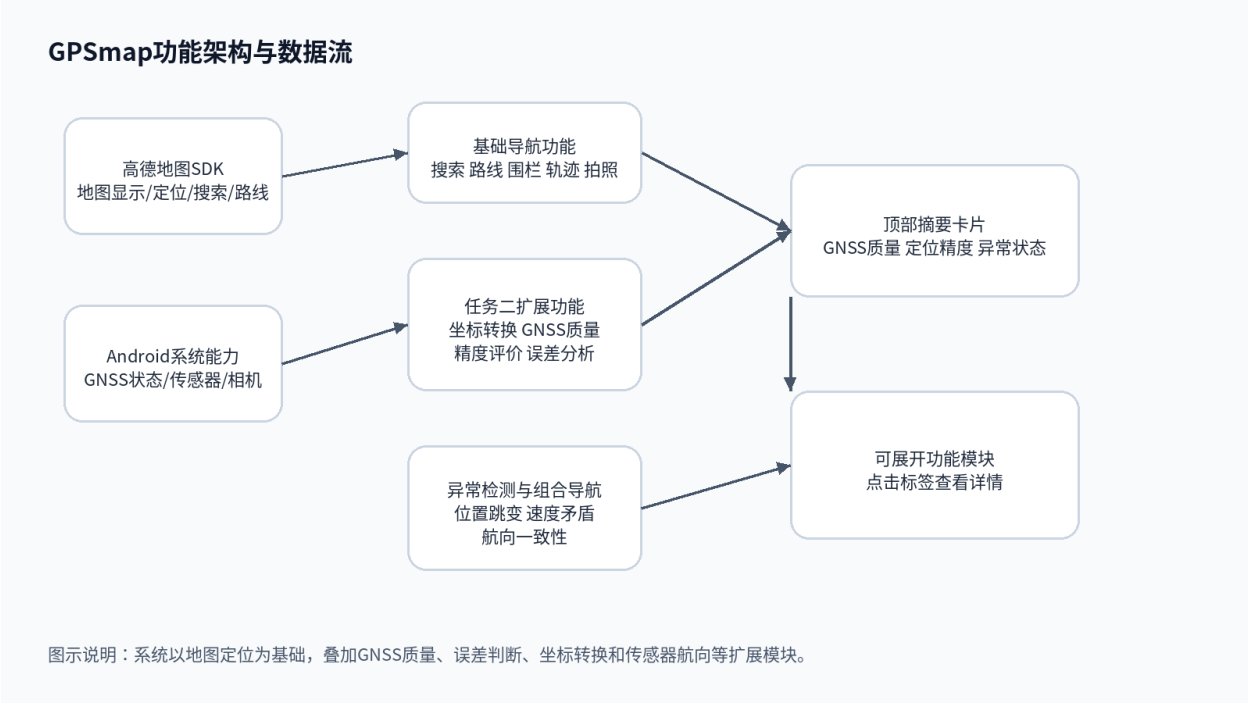


图 1 GPSmap 功能架构与数据流示意图

2.2 坐标转换算法

高德地图定位结果使用 GCJ-02 坐标系。为了满足坐标转换功能，本项目在获得当前位置后同步输出 WGS84 近似坐标和 BD-09 坐标。GCJ-02 到 WGS84 的处理采用常见反算思想：先根据经纬度计算 GCJ-02 相对 WGS84 的经纬度偏移量，再用“原 GCJ-02 坐标减去偏移量”的形式得到 WGS84 近似值。若坐标位于中国大陆偏移适用范围外，则直接返回原始坐标，避免不必要偏移。

BD-09 转换是在 GCJ-02 坐标基础上进行极坐标微扰转换。设 GCJ-02 经纬度为 lon 、 lat ，则先计算 $z = \sqrt{lon^2 + lat^2} + 0.00002 \cdot \sin(lat \cdot \pi \cdot 3000/180)$ ，再计算 $\theta = \text{atan2}(lat, lon) + 0.000003 \cdot \cos(lon \cdot \pi \cdot 3000/180)$ ，最后得到 BD-09 经纬度。该算法适合课程 App 中进行坐标系统展示，可直观说明不同地图平台坐标存在系统差异。

2.3 GNSS 定位质量评估算法

GNSS 定位质量评估主要由卫星数量、信号强度、卫星高度角、星座类型和 DOP 几何因子组成。Android 的 `GnssStatus` 接口能够按卫星逐颗返回状态信息。本项目统计可见卫星总数 `visible`、参与定位卫星数 `used`、平均载噪比 `C/N0`、最大 `C/N0`、平均高度角，并分别统计 GPS、北斗、GLONASS、Galileo 等星座数量。

平均 `C/N0` 计算公式为：平均 `C/N0` = $\sum C/N0_i / n$ ，其中 n 为有效 `C/N0` 大于 0 的卫星数量。一般而言，平均 `C/N0` 越高，说明接收机获得的卫星信号越强；参与定位卫星数越多，说明定位解算约束越充分；卫星高度角越高，通常受遮挡和多路径影响越小。

DOP 反映卫星空间几何构型对定位精度的放大作用。本项目依据卫星高度角 e 和方位角 a 构造近似观测矩阵行向量 $G_i = [\cos(e)\sin(a), \cos(e)\cos(a), \sin(e), 1]$ ，再计算 $Q = (G^T G)^{-1}$ 。当矩阵可逆时，可由 Q 的对角线元素近似得到 GDOP、PDOP、HDOP、VDOP。其中 PDOP 表示三维位置精度因子，HDOP 表示水平位置精度因子，VDOP 表示高程方向精度因子。需要说明的是，手机系统通常不直接提供专业接收机完整观测矩阵，因此本项目 DOP 属于基于卫星几何分布的近似估算，适合教学展示和质量评估。

指标	评价含义	较好情况
可见卫星数	手机当前能够接收到的卫星数量	数量较多，通常室外开阔可达十几颗到数十颗

参与定位卫星数 平均 C/N0	实际参与定位解算的卫星数量 卫星信号强度，单位 dB-Hz	不少于 4 颗，越多越稳定 30dB-Hz 以上一般较好
DOP	卫星几何构型对误差的放大程度	数值越小越好，HDOP/PDOP 小表示几何结构较优
星座数量	GPS、北斗、GLONASS、Galileo 等卫星系统统计	多星座参与通常有助于提高稳定性

2.4 GNSS 定位精度评价算法

定位精度评价直接利用系统定位结果中的水平精度 accuracy，并结合速度、航向和定位方式给出可用性判断。软件将水平精度划分为“优、良、中、差”四个等级：精度 $\leq 5\text{m}$ 为优， $5\sim 15\text{m}$ 为良， $15\sim 30\text{m}$ 为中，超过 30m 为差。该分级不是专业测量规范，而是面向手机导航应用的可读性评价标准，便于用户快速理解当前定位是否适合导航、轨迹记录或围栏判断。

同时，App 会显示速度和航向信息。当速度很低时，GNSS 航向可能不稳定，因为静止或低速状态下由连续位置推算的方向容易受定位噪声影响。因此组合导航模块会结合手机姿态传感器航向进行辅助判断。

2.5 GNSS 导航误差源分析算法

导航误差源分析并不是直接反演每一种误差的物理量，而是根据观测现象推断可能的误差来源。若可见卫星数较少，系统判断存在遮挡或可用卫星不足；若平均 C/N0 较低，判断可能存在信号弱、天线遮挡或室内/半室内环境影响；若 DOP 较大，判断卫星几何构型较差；若定位精度较差但卫星数量较多，则可能存在多路径或融合定位误差。该方法的优势是简单直观，适合手机 App 实时给出解释性结果。

2.6 GNSS 欺骗与异常检测初筛算法

GNSS 欺骗与异常检测属于初筛逻辑，主要检测位置跳变、速度异常和状态矛盾。程序保存上一次定位点和时间，当新定位到来时，计算两次定位之间的球面距离和时间间隔，从而得到推算速度。若短时间内位置跳变距离过大或推算速度明显超过正常行人/车辆范围，而系统给出的速度或卫星状态无法解释该变化，则认为存在疑似异常。

异常检测还会结合卫星状态与定位精度之间的矛盾。例如卫星数极少、平均 C/N0 很低，但系统定位精度却异常乐观时，需要提示结果可能依赖网络融合定位或存在环境不稳定。本功能不能替代专业 GNSS 欺骗检测设备，但可以作为课程 App 中的异常识别演示模块。

2.7 GNSS+惯性传感器组合导航展示算法

组合导航模块使用手机加速度计和磁力计估计手机姿态航向。加速度计提供重力方向，磁力计提供地磁方向，通过 `SensorManager.getRotationMatrix()` 求解旋转矩阵，再由 `SensorManager.getOrientation()` 得到方位角。将方位角转换到 $0\sim 360$ 度范围后，即得到手机传感器航向。

当 GNSS 速度足够大时，GNSS 航向通常具有较好参考意义；当速度较低时，GNSS 航向容易受定位噪声影响。本项目将传感器航向与 GNSS 航向作差，并将角度差归一化到 $0\sim 180$ 度范围。当差值较小时，说明二者一致性较好；当差值较大时，提示可能存在手机姿态变化、磁干扰或 GNSS 航向不稳定。

三、代码

3.1 MainActivity.java

以下为 MainActivity.java 的完整注释版源码：

```
/*
 * GPSmap 主功能代码文件（详细注释版）
 * =====
 * 文件作用：
```

- * 1. 作为 App 的主 Activity，统一管理地图显示、定位回调、POI 搜索、路线规划、电子围栏、轨迹记录、拍照定位等基础功能。
- * 2. 在原有导航定位 App 基础上扩展“任务二”相关功能，包括坐标转换、GNSS 定位质量评估、定位精度评价、导航误差源分析、GNSS 异常/欺骗初筛、GNSS+惯性传感器组合导航展示。
- * 3. 采用“顶部摘要卡片 + 可展开功能模块”的交互方式，避免所有结果文字堆叠在主界面，提高界面清晰度和答辩展示效果。

* 注释阅读建议：

- * - 先看 onCreate()：理解 Activity 启动时如何依次完成隐私合规、布局加载、地图初始化、事件绑定、传感器初始化和权限申请。
- * - 再看 onLocationChanged()：理解定位结果如何驱动界面刷新、围栏判断、轨迹记录和扩展 GNSS 功能更新。
- * - 然后看“任务二功能面板显示控制”“坐标转换”“GNSS 定位质量评估”“异常检测”“传感器融合”等分区。
- * - 最后看搜索、路线、围栏、轨迹和拍照模块，理解原 App 功能如何与新扩展功能共同工作。

* 注意事项：

- * - 本注释版只增加说明性注释，不改变原有代码逻辑。
- * - GNSS 卫星数、C/N0、DOP 等数据依赖真机 GNSS 芯片和室外信号环境，模拟器通常无法提供真实卫星观测数据。

```

*/
package com.example.gpsmap;

// 权限相关常量，例如定位权限、相机权限
import android.Manifest;
// 用于获取系统服务，例如 GNSS 状态服务
import android.content.Context;
// 用于动态改变功能标签按钮的背景颜色
import android.content.res.ColorStateList;
// 用于启动系统组件，例如相机
import android.content.Intent;
// 用于权限检查结果判断
import android.content.pm.PackageManager;
// 用于保存拍照返回的缩略图
import android.graphics.Bitmap;
// 惯性传感器相关类，用于实现 GNSS + 手机传感器辅助航向判断
import android.hardware.Sensor;
import android.hardware.SensorEvent;
import android.hardware.SensorEventListener;
import android.hardware.SensorManager;
// GNSS 卫星状态对象，用于统计可见卫星数、参与定位卫星数、信噪比和卫星几何分布
import android.location.GnssStatus;
// 系统定位服务管理器，用于注册 GNSS 卫星状态回调
import android.location.LocationManager;
// 用于设置文本字体样式
import android.graphics.Typeface;
// 用于动态创建圆角、描边背景
import android.graphics.drawable.GradientDrawable;
// Activity 状态保存与恢复用的 Bundle
import android.os.Bundle;
// 系统相机拍照接口
import android.provider.MediaStore;
// 尺寸单位转换工具
import android.util.TypedValue;
// 控件内容对齐方式
import android.view.Gravity;
// 视图基类
import android.view.View;
// 视图容器相关类
import android.view.ViewGroup;
// 列表适配器基类
import android.widget.BaseAdapter;
// 按钮控件
import android.widget.Button;
// 输入框控件
import android.widget.EditText;
// 图片控件
import android.widget.ImageView;
// 线性布局控件
import android.widget.LinearLayout;
// 列表控件
import android.widget.ListView;
// 滑动条控件
import android.widget.SeekBar;
// 文本控件
import android.widget.TextView;
// 短提示控件
import android.widget.Toast;

// 非空注解
import androidx.annotation.NonNull;
// 对话框
import androidx.appcompat.app.AlertDialog;
// 兼容版 Activity
import androidx.appcompat.app.AppCompatActivity;
// 动态申请权限
import androidx.core.app.ActivityCompat;
// 兼容权限检查
import androidx.core.content.ContextCompat;

// 高德定位结果对象
import com.amap.api.location.AMapLocation;
// 高德定位客户端
import com.amap.api.location.AMapLocationClient;
// 高德定位参数配置
import com.amap.api.location.AMapLocationClientOption;
// 高德定位监听器
import com.amap.api.location.AMapLocationListener;
// 高德地图主对象

```

```

import com.amap.api.maps.AMap;
// 高德地图工具类，主要用于测距
import com.amap.api.maps.AMapUtils;
// 地图镜头更新工厂
import com.amap.api.maps.CameraUpdateFactory;
// 地图定位源接口
import com.amap.api.maps.LocationSource;
// 地图视图控件
import com.amap.api.maps.MapView;
// 高德隐私合规初始化工具
import com.amap.api.maps.MapsInitializer;
// Marker 图标工厂
import com.amap.api.maps.model.BitmapDescriptorFactory;
// 圆形覆盖物对象
import com.amap.api.maps.model.Circle;
// 圆形覆盖物配置
import com.amap.api.maps.model.CircleOptions;
// 经纬度对象
import com.amap.api.maps.model.LatLng;
// 地图范围对象，用于让路线整体显示在屏幕内
import com.amap.api.maps.model.LatLngBounds;
// Marker 对象
import com.amap.api.maps.model.Marker;
// Marker 配置对象
import com.amap.api.maps.model.MarkerOptions;
// 我的位置蓝点样式
import com.amap.api.maps.model.MyLocationStyle;
// 折线对象
import com.amap.api.maps.model.Polyline;
// 折线配置对象
import com.amap.api.maps.model.PolylineOptions;
// 高德服务异常
import com.amap.api.services.core.AMapException;
// 高德服务使用的经纬度点
import com.amap.api.services.core.LatLonPoint;
// POI 单个搜索结果
import com.amap.api.services.core.PoiItem;
// POI 搜索结果集合
import com.amap.api.services.poisearch.PoiResult;
// POI 搜索类
import com.amap.api.services.poisearch.PoiSearch;
// 公交路线结果，这里没有真正使用
import com.amap.api.services.route.BusRouteResult;
// 驾车路径对象
import com.amap.api.services.route.DrivePath;
// 驾车路线结果
import com.amap.api.services.route.DriveRouteResult;
// 骑行路线结果，这里没有真正使用
import com.amap.api.services.route.RideRouteResult;
// 路线搜索主类
import com.amap.api.services.route.RouteSearch;
// 步行路径对象
import com.amap.api.services.route.WalkPath;
// 步行路线结果
import com.amap.api.services.route.WalkRouteResult;
// 步行单步路径
import com.amap.api.services.route.WalkStep;

// 时间格式化
import java.text.SimpleDateFormat;
// 动态数组
import java.util.ArrayList;
// 键值映射
import java.util.HashMap;
// 列表接口
import java.util.List;
// 日期类
import java.util.Date;
// 数学数组工具，用于临时保存 DOP 计算矩阵
import java.util.Arrays;
// 本地化格式工具
import java.util.Locale;

/**
 * 主界面 Activity
 * 负责完成以下功能：
 * 1. 地图显示与定位
 * 2. 搜索地点与路线规划
 * 3. 电子围栏设置与状态判断
 * 4. 轨迹记录与清除
 * 5. 拍照定位与照片查看
 * 6. 坐标转换：GCJ-02、WGS84、BD-09 三套坐标联动显示
 * 7. GNSS 定位质量评估：可见卫星数、参与定位卫星数、C/N0、DOP 估算和质量等级
 * 8. GNSS 定位精度评价：水平精度、定位方式、速度和航向综合判断
 * 9. GNSS 导航误差源分析：根据信号、卫星几何和定位方式推断主要误差来源
 * 10. GNSS 异常/欺骗初筛：检测位置跳变、速度异常和卫星状态矛盾
 * 11. GNSS + 惯性传感器辅助导航：融合 GNSS 航向与手机姿态航向进行一致性判断
 */
public class MainActivity extends AppCompatActivity implements
    LocationSource,
    AMapLocationListener,
    AMap.OnMapLongClickListener,
    AMap.OnMarkerClickListener,

```

```

        PoiSearch.OnPoiSearchListener,
        RouteSearch.OnRouteSearchListener,
        SensorEventListener {

// 定位权限请求码
private static final int REQUEST_LOCATION_PERMISSION = 1001;
// 相机权限请求码
private static final int REQUEST_CAMERA_PERMISSION = 1002;
// 调用系统相机的请求码
private static final int REQUEST_IMAGE_CAPTURE = 2001;

// 地图控件
private MapView mapView;
// 高德地图对象
private AMap aMap;

// 搜索关键字输入框
private EditText etKeyword;
// 搜索结果列表
private ListView lvSearchResults;

// 地址显示文本
private TextView tvAddress;
// 经纬度与精度显示文本
private TextView tvCoord;
// 坐标转换结果显示文本
private TextView tvCoordConverted;
// GNSS 定位质量评估显示文本
private TextView tvGnssQuality;
// GNSS 定位精度评价显示文本
private TextView tvAccuracyEval;
// GNSS 导航误差源分析显示文本
private TextView tvErrorAnalysis;
// GNSS 异常与欺骗初筛显示文本
private TextView tvAnomalyDetect;
// GNSS + 惯性传感器辅助导航显示文本
private TextView tvSensorFusion;
// 顶部摘要卡片: GNSS 质量摘要
private TextView tvQuickGnss;
// 顶部摘要卡片: 定位精度摘要
private TextView tvQuickAccuracy;
// 顶部摘要卡片: 异常检测摘要
private TextView tvQuickAnomaly;
// 当前选中的任务二功能标题
private TextView tvFeatureTitle;
// 当前选中的任务二功能详情
private TextView tvFeatureContent;
// 功能切换按钮: 坐标转换
private Button btnShowCoord;
// 功能切换按钮: 卫星质量
private Button btnShowGnss;
// 功能切换按钮: 精度评价
private Button btnShowAccuracy;
// 功能切换按钮: 误差源分析
private Button btnShowError;
// 功能切换按钮: 异常检测
private Button btnShowAnomaly;
// 功能切换按钮: 组合导航
private Button btnShowSensor;
// 功能模块内容区, 默认收起, 点击“展开”后显示具体功能按钮和详情卡片
private LinearLayout layoutFeatureModuleContent;
// 功能模块展开收起按钮
private Button btnToggleFeatureModule;
// 功能模块是否展开
private boolean isFeatureModuleExpanded = false;
// 当前详情卡片正在显示的功能类型
private String activeFeaturePanel = "gnss";
// 围栏状态显示文本
private TextView tvFenceStatus;
// 轨迹状态显示文本
private TextView tvTrackStatus;
// 路线状态显示文本
private TextView tvRouteStatus;
// 围栏半径显示文本
private TextView tvFenceRadius;
// 已选目的地显示文本
private TextView tvSelectedDestination;

// 顶部可展开的信息区
private LinearLayout layoutSearchExtras;
// 路线按钮区域
private LinearLayout layoutRouteActions;
// 围栏半径布局
private LinearLayout layoutFenceRadius;
// 围栏半径卡片
private View cardFenceRadius;

// 顶部展开收起按钮
private Button btnToggleInfo;
// 搜索按钮
private Button btnSearch;
// 步行路线按钮
private Button btnWalkRoute;

```

```

// 驾车路线按钮
private Button btnDriveRoute;
// 清除路线按钮
private Button btnClearRoute;

// 围栏按钮
private Button btnFenceMode;
// 清除围栏按钮
private Button btnClearFence;

// 拍照按钮
private Button btnTakePhoto;
// 清除拍照点按钮
private Button btnClearPhotos;

// 轨迹按钮
private Button btnTrack;
// 清除轨迹按钮
private Button btnClearTrack;

// 回到当前位置按钮
private Button btnCenterMe;

// 围栏半径滑动条
private SeekBar seekFenceRadius;

// 顶部信息区是否展开
private boolean isInfoExpanded = true;

// 地图蓝点定位回调监听器
private OnLocationChangeListener mListener;
// 高德定位客户端
private AMapLocationClient aMapLocationClient;
// 定位参数配置
private AMapLocationClientOption aMapLocationClientOption;

// 系统 GNSS 状态服务，用于读取原始卫星状态
private LocationManager locationManager;
// GNSS 卫星状态回调
private GnssStatus.Callback gnssStatusCallback;
// 是否已经注册 GNSS 监听，避免重复注册
private boolean isGnssStatusRegistered = false;
// 当前可见卫星数
private int gnssVisibleCount = 0;
// 当前参与定位解算的卫星数
private int gnssUsedCount = 0;
// 当前平均载噪比 C/N0，单位 dB-Hz
private double gnssAvgCn0 = 0.0;
// 当前最大载噪比 C/N0，单位 dB-Hz
private double gnssMaxCn0 = 0.0;
// 当前平均卫星高度角，单位度
private double gnssAvgElevation = 0.0;
// 估算 GDOP、PDOP、HDOP、VDOP
private double gnssGdop = Double.NaN;
private double gnssPdop = Double.NaN;
private double gnssHdop = Double.NaN;
private double gnssVdop = Double.NaN;
// 各星座卫星数量统计
private int gpsCount = 0;
private int beidouCount = 0;
private int glonassCount = 0;
private int galileoCount = 0;
// GNSS 综合质量等级
private String gnssQualityLevel = "等待卫星数据";

// 系统传感器管理器，用于读取加速度计和磁力计
private SensorManager sensorManager;
// 加速度计传感器
private Sensor accelerometerSensor;
// 磁力计传感器
private Sensor magneticSensor;
// 加速度计最近一次读数
private final float[] accelerometerValues = new float[3];
// 磁力计最近一次读数
private final float[] magneticValues = new float[3];
// 是否已经获得加速度计读数
private boolean hasAccelerometerValue = false;
// 是否已经获得磁力计读数
private boolean hasMagneticValue = false;
// 由惯性/磁传感器估计的手机航向角，单位度
private float sensorHeadingDeg = Float.NaN;
// 当前 GNSS 速度，单位 m/s
private float currentGnssSpeed = 0f;
// 当前 GNSS 航向角，单位度
private float currentGnssBearing = Float.NaN;
// 当前水平定位精度，单位 m，用于顶部摘要卡片
private float currentHorizontalAccuracy = -1f;
// 当前精度等级，用于顶部摘要卡片
private String currentAccuracyLevel = "等待定位";
// 上一次用于异常检测的位置
private LatLng lastAnomalyLatLng;
// 上一次用于异常检测的时间
private long lastAnomalyTimeMillis = 0L;
// 最近两次定位间的位置跳变量，单位 m

```

```

private double lastJumpDistance = 0.0;
// 最近两次定位间推算的移动速度, 单位 m/s
private double lastJumpSpeed = 0.0;
// 最近一次异常检测结论
private String anomalyLevel = "待检测";
// 最近一次异常检测原因
private String anomalyReason = "等待连续定位数据";

// 是否是首次定位
private boolean isFirstLocate = true;
// 当前经纬度
private LatLng currentLatLng;
// 当前地址
private String currentAddress = "未知地址";
// 当前城市
private String currentCity = "";

// 围栏模式是否开启
private boolean isFenceModeEnabled = false;
// 围栏中心点
private LatLng fenceCenter;
// 围栏圆对象
private Circle fenceCircle;
// 围栏中心 Marker
private Marker fenceMarker;
// 围栏半径, 默认 100 米
private float fenceRadius = 100f;
// 上一次是否在围栏内, 用于判断进入和离开
private Boolean lastInsideState = null;

// 是否正在记录轨迹
private boolean isTracking = false;
// 轨迹点集合
private final ArrayList<LatLng> trackPoints = new ArrayList<>();
// 轨迹折线对象
private Polyline trackPolyline;
// 轨迹累计长度
private float trackDistance = 0f;

// 路线搜索对象
private RouteSearch routeSearch;
// 目的地 Marker
private Marker destinationMarker;
// 目的地坐标
private LatLng routeDestination;
// 目的地名称
private String routeDestinationName = "";
// 路线折线
private Polyline routePolyline;

// 搜索结果集合
private final ArrayList<PoiItem> searchPoiList = new ArrayList<>();
// 搜索结果适配器
private SearchResultAdapter searchAdapter;

// 拍照前暂存的拍照点位置
private LatLng pendingPhotoLatLng;
// 拍照前暂存的时间
private String pendingPhotoTime;
// 拍照前暂存的地址
private String pendingPhotoAddress;
// 用 Marker 的 id 关联 Bitmap
private final HashMap<String, Bitmap> photoBitmapMap = new HashMap<>();
// 用 Marker 的 id 关联拍照说明信息
private final HashMap<String, String> photoInfoMap = new HashMap<>();
// 所有拍照点 Marker 的集合, 便于统一清除
private final ArrayList<Marker> photoMarkers = new ArrayList<>();
// 时间格式化器
private final SimpleDateFormat timeFormat =
    new SimpleDateFormat("yyyy-MM-dd HH:mm:ss", Locale.getDefault());

/**
 * Activity 创建入口
 * 完成布局加载、地图初始化、事件绑定、权限申请等工作
 */
@Override
protected void onCreate(Bundle savedInstanceState) {
    super.onCreate(savedInstanceState);

    // 隐藏默认标题栏, 让地图界面更完整
    if (getSupportActionBar() != null) {
        getSupportActionBar().hide();
    }

    // 高德地图与定位 SDK 的隐私合规初始化
    MapsInitializer.updatePrivacyShow(this, true, true);
    MapsInitializer.updatePrivacyAgree(this, true);
    AMapLocationClient.updatePrivacyShow(this, true, true);
    AMapLocationClient.updatePrivacyAgree(this, true);

    // 加载主布局文件
    setContentView(R.layout.activity_main);

    // 绑定界面控件

```

```

initViews();

// 初始化 MapView 生命周期
mapView.onCreate(savedInstanceState);

// 获取 AMap 对象
if (aMap == null) {
    aMap = mapView.getMap();
}

try {
    // 创建路线搜索对象
    routeSearch = new RouteSearch(this);
    // 设置路线回调监听器
    routeSearch.setRouteSearchListener(this);
} catch (AMapException e) {
    e.printStackTrace();
}

// 初始化地图样式与地图监听
initMap();
// 初始化按钮和控件事件
initEvents();
// 初始化惯性传感器监听对象，用于 GNSS + 惯性辅助导航展示
initSensorFusion();
// 检查并申请定位权限
requestLocationPermissionIfNeeded();

// 初始化顶部状态文字
updateFenceStatusNoFence();
updateTrackStatus();
updateGnssQualityText();
updateAccuracyEvalText(null);
updateErrorAnalysisText(null);
updateAnomalyDetectText();
updateSensorFusionText();
updateQuickStatusCards(null);
refreshFeaturePanel();
tvRouteStatus.setText("路线：未规划");
}

/**
 * 绑定布局中的所有控件
 */
private void initViews() {
    mapView = findViewById(R.id.mapView);

    etKeyword = findViewById(R.id.etKeyword);
    lvSearchResults = findViewById(R.id.lvSearchResults);

    tvAddress = findViewById(R.id.tvAddress);
    tvCoord = findViewById(R.id.tvCoord);
    tvCoordConverted = findViewById(R.id.tvCoordConverted);
    tvGnssQuality = findViewById(R.id.tvGnssQuality);
    tvAccuracyEval = findViewById(R.id.tvAccuracyEval);
    tvErrorAnalysis = findViewById(R.id.tvErrorAnalysis);
    tvAnomalyDetect = findViewById(R.id.tvAnomalyDetect);
    tvSensorFusion = findViewById(R.id.tvSensorFusion);
    tvQuickGnss = findViewById(R.id.tvQuickGnss);
    tvQuickAccuracy = findViewById(R.id.tvQuickAccuracy);
    tvQuickAnomaly = findViewById(R.id.tvQuickAnomaly);
    tvFeatureTitle = findViewById(R.id.tvFeatureTitle);
    tvFeatureContent = findViewById(R.id.tvFeatureContent);
    tvFenceStatus = findViewById(R.id.tvFenceStatus);
    tvTrackStatus = findViewById(R.id.tvTrackStatus);
    tvRouteStatus = findViewById(R.id.tvRouteStatus);
    tvFenceRadius = findViewById(R.id.tvFenceRadius);
    tvSelectedDestination = findViewById(R.id.tvSelectedDestination);

    layoutSearchExtras = findViewById(R.id.layoutSearchExtras);
    layoutFeatureModuleContent = findViewById(R.id.layoutFeatureModuleContent);
    layoutRouteActions = findViewById(R.id.layoutRouteActions);
    layoutFenceRadius = findViewById(R.id.layoutFenceRadius);
    cardFenceRadius = findViewById(R.id.cardFenceRadius);

    btnToggleInfo = findViewById(R.id.btnToggleInfo);
    btnToggleFeatureModule = findViewById(R.id.btnToggleFeatureModule);
    btnSearch = findViewById(R.id.btnSearch);
    btnWalkRoute = findViewById(R.id.btnWalkRoute);
    btnDriveRoute = findViewById(R.id.btnDriveRoute);
    btnClearRoute = findViewById(R.id.btnClearRoute);

    btnFenceMode = findViewById(R.id.btnFenceMode);
    btnClearFence = findViewById(R.id.btnClearFence);

    btnTakePhoto = findViewById(R.id.btnTakePhoto);
    btnClearPhotos = findViewById(R.id.btnClearPhotos);

    btnTrack = findViewById(R.id.btnTrack);
    btnClearTrack = findViewById(R.id.btnClearTrack);

    btnShowCoord = findViewById(R.id.btnShowCoord);
    btnShowGnss = findViewById(R.id.btnShowGnss);
    btnShowAccuracy = findViewById(R.id.btnShowAccuracy);
    btnShowError = findViewById(R.id.btnShowError);
    btnShowAnomaly = findViewById(R.id.btnShowAnomaly);
    btnShowSensor = findViewById(R.id.btnShowSensor);

```

```

        btnCenterMe = findViewById(R.id.btnCenterMe);

        seekFenceRadius = findViewById(R.id.seekFenceRadius);

        // 创建并设置搜索结果适配器
        searchAdapter = new SearchResultAdapter();
        lvSearchResults.setAdapter(searchAdapter);
    }

    /**
     * 初始化地图外观与地图事件监听
     */
    private void initMap() {
        // 开启比例尺
        aMap.getUiSettings().setScaleControlsEnabled(true);
        // 关闭默认定位按钮, 使用自定义按钮
        aMap.getUiSettings().setMyLocationButtonEnabled(false);
        // 关闭默认缩放按钮
        aMap.getUiSettings().setZoomControlsEnabled(false);

        // 地图长按监听, 用于设置围栏中心
        aMap.setOnMapLongClickListener(this);
        // Marker 点击监听, 用于拍照点击查看
        aMap.setOnMarkerClickListener(this);

        // 自定义定位蓝点样式
        MyLocationStyle myLocationStyle = new MyLocationStyle();
        // 只旋转蓝点, 不强制地图始终居中
        myLocationStyle.myLocationType(MyLocationStyle.LOCATION_TYPE_LOCATION_ROTATE_NO_CENTER);
        // 定位刷新间隔 2 秒
        myLocationStyle.interval(2000);
        // 蓝圈填充颜色
        myLocationStyle.radiusFillColor(0x180EA5E9);
        // 蓝圈边框颜色
        myLocationStyle.strokeColor(0x440EA5E9);
        // 蓝圈边框宽度
        myLocationStyle.strokeWidth(1f);
        // 应用蓝点样式
        aMap.setMyLocationStyle(myLocationStyle);
    }

    /**
     * 初始化所有控件事件
     */
    private void initEvents() {
        // SeekBar 最大值 450, 对应实际半径 50 到 500 米
        seekFenceRadius.setMax(450);
        // 默认进度 50, 对应半径 100 米
        seekFenceRadius.setProgress(50);
        tvFenceRadius.setText("围栏半径: 100m");

        // 围栏半径滑动监听
        seekFenceRadius.setOnSeekBarChangeListener(new SeekBar.OnSeekBarChangeListener() {
            @Override
            public void onProgressChanged(SeekBar seekBar, int progress, boolean fromUser) {
                // 实际半径 = progress + 50
                fenceRadius = progress + 50;
                tvFenceRadius.setText("围栏半径: " + (int) fenceRadius + "m");

                // 如果地图上已经有围栏圆, 则同步更新半径
                if (fenceCircle != null) {
                    fenceCircle.setRadius(fenceRadius);
                }

                // 如果当前位置和围栏中心都存在, 则实时重新判断围栏状态
                if (currentLatLng != null && fenceCenter != null) {
                    checkFenceState(currentLatLng, false);
                }
            }

            @Override
            public void onStartTrackingTouch(SeekBar seekBar) {}

            @Override
            public void onStopTrackingTouch(SeekBar seekBar) {}
        });

        // 展开收起顶部信息区
        btnToggleInfo.setOnClickListener(v -> toggleInfoPanel());

        // 展开收起功能模块详情区, 默认只保留摘要卡片, 降低顶部展示栏高度
        btnToggleFeatureModule.setOnClickListener(v -> toggleFeatureModule());

        // 搜索地点
        btnSearch.setOnClickListener(v -> searchLocation());

        // 点击某条搜索结果后选中目的地
        lvSearchResults.setOnItemClickListener((parent, view, position, id) -> selectSearchResult(position));

        // 步行路线规划
        btnWalkRoute.setOnClickListener(v -> planWalkRoute());
        // 驾车路线规划
        btnDriveRoute.setOnClickListener(v -> planDriveRoute());
        // 清除路线
        btnClearRoute.setOnClickListener(v -> clearRoute());
    }

```

```

// 任务二功能详情切换按钮：默认只展示一个功能详情，避免顶部出现大段文字
btnShowCoord.setOnClickListener(v -> showFeaturePanel("coord"));
btnShowGnss.setOnClickListener(v -> showFeaturePanel("gnss"));
btnShowAccuracy.setOnClickListener(v -> showFeaturePanel("accuracy"));
btnShowError.setOnClickListener(v -> showFeaturePanel("error"));
btnShowAnomaly.setOnClickListener(v -> showFeaturePanel("anomaly"));
btnShowSensor.setOnClickListener(v -> showFeaturePanel("sensor"));
showFeaturePanel("gnss");

// 围栏模式开关
btnFenceMode.setOnClickListener(v -> toggleFenceMode());
// 清除围栏
btnClearFence.setOnClickListener(v -> clearFence());

// 拍照定位
btnTakePhoto.setOnClickListener(v -> takePhotoWithLocation());
// 清除拍照点
btnClearPhotos.setOnClickListener(v -> clearPhotoMarkers());

// 轨迹记录开关
btnTrack.setOnClickListener(v -> toggleTrackRecording());
// 清除轨迹
btnClearTrack.setOnClickListener(v -> clearTrack());

// 回到当前位置
btnCenterMe.setOnClickListener(v -> {
    if (currentLatLng != null) {
        // 有定位时把地图移动到当前位置
        aMap.animateCamera(CameraUpdateFactory.newLatLngZoom(currentLatLng, 18f));
    } else {
        // 没拿到定位时给提示
        Toast.makeText(this, "当前位置未获取到", Toast.LENGTH_SHORT).show();
    }
});
}

/**
 * 切换顶部信息区的展开和收起状态
 */
private void toggleInfoPanel() {
    isInfoExpanded = !isInfoExpanded;
    layoutSearchExtras.setVisibility(isInfoExpanded ? View.VISIBLE : View.GONE);
    btnToggleInfo.setText(isInfoExpanded ? "收起" : "展开");
}

/**
 * 展开或收起功能模块区。
 * 默认状态下只保留三项摘要卡片，点击展开后再显示具体功能标签和详情卡片。
 */
private void toggleFeatureModule() {
    isFeatureModuleExpanded = !isFeatureModuleExpanded;
    layoutFeatureModuleContent.setVisibility(isFeatureModuleExpanded ? View.VISIBLE : View.GONE);
    btnToggleFeatureModule.setText(isFeatureModuleExpanded ? "收起" : "展开");
}

// ===== 任务二功能面板显示控制 =====

/**
 * 切换任务二详情卡片展示内容。顶部只保留摘要，具体功能通过标签按钮按展开。
 */
private void showFeaturePanel(String featureName) {
    if (featureName != null && !featureName.trim().isEmpty()) {
        activeFeaturePanel = featureName;
    }
    updateFeatureButtonStyle();
    refreshFeaturePanel();
}

/**
 * 根据当前选择的功能，把对应文字放入统一详情卡片。
 */
private void refreshFeaturePanel() {
    if (tvFeatureTitle == null || tvFeatureContent == null) {
        return;
    }

    String title;
    String content;
    switch (activeFeaturePanel) {
        case "coord":
            title = "坐标转换";
            content = getTextFromView(tvCoordConverted, "坐标转换：等待定位数据");
            break;
        case "accuracy":
            title = "GNSS 定位精度评价";
            content = getTextFromView(tvAccuracyEval, "定位精度评价：等待定位数据");
            break;
        case "error":
            title = "GNSS 导航误差源分析";
            content = getTextFromView(tvErrorAnalysis, "误差源分析：等待定位数据");
            break;
        case "anomaly":
            title = "GNSS 欺骗与异常检测初筛";
            content = getTextFromView(tvAnomalyDetect, "异常/欺骗初筛：待检测");
            break;
        case "sensor":

```

```

        title = "GNSS+惯性传感器组合导航";
        content = getTextFromView(tvSensorFusion, "组合导航: 等待传感器数据");
        break;
    case "gnss":
    default:
        title = "GNSS 定位质量评估";
        content = getTextFromView(tvGnssQuality, "GNSS 质量: 等待卫星数据");
        break;
    }

    tvFeatureTitle.setText(title);
    tvFeatureContent.setText(content);
}

/**
 * 让当前选中的功能标签更醒目, 其余标签保持浅色。
 */
private void updateFeatureButtonStyle() {
    setFeatureButtonSelected(btnShowGnss, "gnss".equals(activeFeaturePanel));
    setFeatureButtonSelected(btnShowAccuracy, "accuracy".equals(activeFeaturePanel));
    setFeatureButtonSelected(btnShowCoord, "coord".equals(activeFeaturePanel));
    setFeatureButtonSelected(btnShowError, "error".equals(activeFeaturePanel));
    setFeatureButtonSelected(btnShowAnomaly, "anomaly".equals(activeFeaturePanel));
    setFeatureButtonSelected(btnShowSensor, "sensor".equals(activeFeaturePanel));
}

/**
 * 设置功能标签按钮的选中/未选中样式。
 */
private void setFeatureButtonSelected(Button button, boolean selected) {
    if (button == null) {
        return;
    }
    button.setTextColor(selected ? 0xFFFFFFFF : 0xFF334155);
    button.setBackgroundTintList(ColorStateList.valueOf(selected ? 0xFF475569 : 0xFFE2E8F0));
}

/**
 * 安全读取隐藏 TextView 中的功能结果。
 */
private String getTextFromView(TextView textView, String defaultText) {
    if (textView == null || textView.getText() == null || textView.getText().toString().trim().isEmpty()) {
        return defaultText;
    }
    return textView.getText().toString();
}

/**
 * 刷新顶部三个摘要卡片, 只显示最关键结论, 避免主界面信息过载。
 */
private void updateQuickStatusCards(AMapLocation location) {
    if (location != null) {
        currentHorizontalAccuracy = location.getAccuracy();
        currentAccuracyLevel = evaluateAccuracyLevel(currentHorizontalAccuracy);
    }

    if (tvQuickGnss != null) {
        if (gnssVisibleCount == 0) {
            tvQuickGnss.setText("等待卫星");
            tvQuickGnss.setTextColor(0xFF64748B);
        } else {
            tvQuickGnss.setText(String.format(
                Locale.getDefault(),
                "%s %d/%d 颗",
                gnssQualityLevel,
                gnssUsedCount,
                gnssVisibleCount
            ));
            tvQuickGnss.setTextColor(getQualityColor(gnssQualityLevel));
        }
    }

    if (tvQuickAccuracy != null) {
        if (currentHorizontalAccuracy < 0f) {
            tvQuickAccuracy.setText("等待定位");
            tvQuickAccuracy.setTextColor(0xFF64748B);
        } else {
            tvQuickAccuracy.setText(String.format(
                Locale.getDefault(),
                "%s %.1fm",
                currentAccuracyLevel,
                currentHorizontalAccuracy
            ));
            tvQuickAccuracy.setTextColor(getAccuracyColor(currentHorizontalAccuracy));
        }
    }

    if (tvQuickAnomaly != null) {
        tvQuickAnomaly.setText(anomalyLevel == null ? "待检测" : anomalyLevel);
        tvQuickAnomaly.setTextColor(getAnomalyColor(anomalyLevel));
    }
}

/**
 * 根据 GNSS 质量等级设置摘要文字颜色。
 */
private int getQualityColor(String level) {
    if ("优".equals(level) || "良".equals(level)) {

```

```

        return 0xFF166534;
    }
    if ("中".equals(level)) {
        return 0xFF92400E;
    }
    if ("差".equals(level)) {
        return 0xFFB45309;
    }
    return 0xFF64748B;
}

/**
 * 根据系统水平精度设置摘要文字颜色。
 */
private int getAccuracyColor(float accuracy) {
    if (accuracy <= 15f) {
        return 0xFF166534;
    }
    if (accuracy <= 30f) {
        return 0xFF92400E;
    }
    return 0xFFB45309;
}

/**
 * 根据异常检测等级设置摘要文字颜色。
 */
private int getAnomalyColor(String level) {
    if ("正常".equals(level)) {
        return 0xFF166534;
    }
    if ("轻微异常".equals(level)) {
        return 0xFF92400E;
    }
    if ("疑似异常".equals(level)) {
        return 0xFFB91C1C;
    }
    return 0xFF64748B;
}

/**
 * 检查是否已有定位权限，没有则申请
 */
private void requestLocationPermissionIfNeeded() {
    if (ContextCompat.checkSelfPermission(this, Manifest.permission.ACCESS_FINE_LOCATION)
        == PackageManager.PERMISSION_GRANTED) {
        enableLocation();
    } else {
        ActivityCompat.requestPermissions(
            this,
            new String[]{
                Manifest.permission.ACCESS_FINE_LOCATION,
                Manifest.permission.ACCESS_COARSE_LOCATION
            },
            REQUEST_LOCATION_PERMISSION
        );
    }
}

/**
 * 启用地图定位
 */
private void enableLocation() {
    aMap.setLocationSource(this);
    aMap.setMyLocationEnabled(true);
    // 同步启动 GNSS 原始卫星状态监听，用于完成任务二中的定位质量评估
    startGnssStatusMonitor();
}

/**
 * 地图需要定位时会调用这里
 * 初始化定位客户端并启动定位
 */
@Override
public void activate(OnLocationChangeListener onLocationChangeListener) {
    mListener = onLocationChangeListener;

    if (aMapLocationClient == null) {
        try {
            aMapLocationClient = new AMapLocationClient(getApplicationContext());
        } catch (Exception e) {
            e.printStackTrace();
            return;
        }
    }

    aMapLocationClientOption = new AMapLocationClientOption();
    aMapLocationClient.setLocationListener(this);

    // 使用高精度定位模式
    aMapLocationClientOption.setLocationMode(
        AMapLocationClientOption.AMapLocationMode.Hight_Accuracy
    );
    // 需要返回地址信息
    aMapLocationClientOption.setNeedAddress(true);
    // 持续定位
    aMapLocationClientOption.setOnceLocation(false);
    // 定位间隔 2 秒
    aMapLocationClientOption.setInterval(2000);
}

```

```

        aMapLocationClient.setLocationOption(aMapLocationClientOption);
    }

    if (aMapLocationClient != null) {
        aMapLocationClient.startLocation();
    }
}

/**
 * 地图不再需要定位时调用
 */
@Override
public void deactivate() {
    mListener = null;

    if (aMapLocationClient != null) {
        aMapLocationClient.stopLocation();
        aMapLocationClient.onDestroy();
        aMapLocationClient = null;
    }
    stopGnssStatusMonitor();
}

/**
 * 定位结果回调
 * 每次定位刷新都会进入这里
 */
@Override
public void onLocationChanged(AMapLocation aMapLocation) {
    if (mListener != null && aMapLocation != null) {
        // errorCode 为 0 代表定位成功
        if (aMapLocation.getErrorCode() == 0) {
            // 把定位结果传给地图蓝点层
            mListener.onLocationChanged(aMapLocation);

            // 更新当前位置
            currentLatLng = new LatLng(aMapLocation.getLatitude(), aMapLocation.getLongitude());
            // 更新地址
            currentAddress = aMapLocation.getAddress() == null ? "未知地址" : aMapLocation.getAddress();
            // 更新城市
            currentCity = aMapLocation.getCity() == null ? "" : aMapLocation.getCity();

            // 刷新顶部定位信息
            updateLocationInfo(aMapLocation);
            // 刷新定位精度评价、误差源分析、异常检测和组合导航状态
            updateExtendedGnssFunctions(aMapLocation);

            // 首次定位成功后自动移动到当前位置
            if (isFirstLocate && currentLatLng != null) {
                aMap.animateCamera(CameraUpdateFactory.newLatLngZoom(currentLatLng, 17f));
                isFirstLocate = false;
            }

            // 每次定位更新都重新判断围栏状态
            checkFenceState(currentLatLng, true);

            // 如果正在记录轨迹，则加入新点
            if (isTracking) {
                addTrackPoint(currentLatLng);
            }
        }
    }
}

/**
 * 更新地址、经纬度、精度和坐标转换结果显示
 */
private void updateLocationInfo(AMapLocation aMapLocation) {
    double gcjLat = aMapLocation.getLatitude();
    double gcjLon = aMapLocation.getLongitude();
    float accuracy = aMapLocation.getAccuracy();

    tvAddress.setText("地址: " + currentAddress);
    tvCoord.setText(String.format(
        Locale.getDefault(),
        "GCJ-02: 纬度 %.6f 经度 %.6f 精度 %.1fm | 精度等级: %s",
        gcjLat,
        gcjLon,
        accuracy,
        evaluateAccuracyLevel(accuracy)
    ));

    updateCoordinateConversionText(gcjLat, gcjLon);
}

// ===== 坐标转换 =====

/**
 * 根据高德地图返回的 GCJ-02 坐标，换算 WGS84 和 BD-09 坐标并显示
 */
private void updateCoordinateConversionText(double gcjLat, double gcjLon) {
    double[] wgs84 = gcj02ToWgs84(gcjLat, gcjLon);
    double[] bd09 = gcj02ToBd09(gcjLat, gcjLon);

    tvCoordConverted.setText(String.format(
        Locale.getDefault(),

```

```

        "坐标转换: WGS84[%f, %f] BD-09[%f, %f]",
        wgs84[0],
        wgs84[1],
        bd09[0],
        bd09[1]
    ));
    if ("coord".equals(activeFeaturePanel)) {
        refreshFeaturePanel();
    }
}

/**
 * 将 GCJ-02 坐标近似反算为 WGS84 坐标
 */
private double[] gcj02ToWgs84(double lat, double lon) {
    if (outOfChina(lat, lon)) {
        return new double[]{lat, lon};
    }

    double[] delta = transformLatLon(lat, lon);
    double dLat = delta[0];
    double dLon = delta[1];
    double mgLat = lat + dLat;
    double mgLon = lon + dLon;

    return new double[]{lat * 2 - mgLat, lon * 2 - mgLon};
}

/**
 * 将 GCJ-02 坐标转换为百度地图常用的 BD-09 坐标
 */
private double[] gcj02ToBd09(double lat, double lon) {
    double x = lon;
    double y = lat;
    double z = Math.sqrt(x * x + y * y) + 0.00002 * Math.sin(y * Math.PI * 3000.0 / 180.0);
    double theta = Math.atan2(y, x) + 0.00003 * Math.cos(x * Math.PI * 3000.0 / 180.0);
    double bdLon = z * Math.cos(theta) + 0.0065;
    double bdLat = z * Math.sin(theta) + 0.006;
    return new double[]{bdLat, bdLon};
}

/**
 * GCJ-02 与 WGS84 之间的经纬度偏移计算
 */
private double[] transformLatLon(double lat, double lon) {
    double a = 6378245.0;
    double ee = 0.00669342162296594323;
    double dLat = transformLat(lon - 105.0, lat - 35.0);
    double dLon = transformLon(lon - 105.0, lat - 35.0);
    double radLat = lat / 180.0 * Math.PI;
    double magic = Math.sin(radLat);
    magic = 1 - ee * magic * magic;
    double sqrtMagic = Math.sqrt(magic);
    dLat = (dLat * 180.0) / ((a * (1 - ee)) / (magic * sqrtMagic) * Math.PI);
    dLon = (dLon * 180.0) / (a / sqrtMagic * Math.cos(radLat) * Math.PI);
    return new double[]{dLat, dLon};
}

/**
 * 纬度方向偏移函数
 */
private double transformLat(double x, double y) {
    double ret = -100.0 + 2.0 * x + 3.0 * y + 0.2 * y * y
        + 0.1 * x * y + 0.2 * Math.sqrt(Math.abs(x));
    ret += (20.0 * Math.sin(6.0 * x * Math.PI)
        + 20.0 * Math.sin(2.0 * x * Math.PI)) * 2.0 / 3.0;
    ret += (20.0 * Math.sin(y * Math.PI)
        + 40.0 * Math.sin(y / 3.0 * Math.PI)) * 2.0 / 3.0;
    ret += (160.0 * Math.sin(y / 12.0 * Math.PI)
        + 320 * Math.sin(y * Math.PI / 30.0)) * 2.0 / 3.0;
    return ret;
}

/**
 * 经度方向偏移函数
 */
private double transformLon(double x, double y) {
    double ret = 300.0 + x + 2.0 * y + 0.1 * x * x
        + 0.1 * x * y + 0.1 * Math.sqrt(Math.abs(x));
    ret += (20.0 * Math.sin(6.0 * x * Math.PI)
        + 20.0 * Math.sin(2.0 * x * Math.PI)) * 2.0 / 3.0;
    ret += (20.0 * Math.sin(x * Math.PI)
        + 40.0 * Math.sin(x / 3.0 * Math.PI)) * 2.0 / 3.0;
    ret += (150.0 * Math.sin(x / 12.0 * Math.PI)
        + 300.0 * Math.sin(x / 30.0 * Math.PI)) * 2.0 / 3.0;
    return ret;
}

/**
 * 判断坐标是否位于中国大陆坐标偏移适用范围外
 */
private boolean outOfChina(double lat, double lon) {
    return lon < 72.004 || lon > 137.8347 || lat < 0.8293 || lat > 55.8271;
}

/**
 * 根据系统定位精度给出简明等级, 便于课堂展示 GNSS 定位精度评价
 */
private String evaluateAccuracyLevel(float accuracy) {

```

```

        if (accuracy <= 5f) {
            return "优";
        } else if (accuracy <= 15f) {
            return "良";
        } else if (accuracy <= 30f) {
            return "中";
        } else {
            return "差";
        }
    }
}

// ===== GNSS 定位质量评估 =====

/**
 * 启动 GNSS 卫星状态监听, 读取可见卫星数、参与定位卫星数、C/N0 和卫星几何信息
 */
private void startGnssStatusMonitor() {
    if (ContextCompat.checkSelfPermission(this, Manifest.permission.ACCESS_FINE_LOCATION)
        != PackageManager.PERMISSION_GRANTED) {
        return;
    }

    if (locationManager == null) {
        locationManager = (LocationManager) getSystemService(Context.LOCATION_SERVICE);
    }

    if (locationManager == null || !isGnssStatusRegistered) {
        return;
    }

    if (gnssStatusCallback == null) {
        gnssStatusCallback = new GnssStatus.Callback() {
            @Override
            public void onSatelliteStatusChanged(@NonNull GnssStatus status) {
                updateGnssStatus(status);
            }
        };
    }

    try {
        isGnssStatusRegistered = locationManager.registerGnssStatusCallback(gnssStatusCallback);
        updateGnssQualityText();
    } catch (SecurityException e) {
        isGnssStatusRegistered = false;
        e.printStackTrace();
    }
}

/**
 * 停止 GNSS 卫星状态监听, 避免 Activity 销毁后继续回调
 */
private void stopGnssStatusMonitor() {
    if (locationManager != null && gnssStatusCallback != null && isGnssStatusRegistered) {
        locationManager.unregisterGnssStatusCallback(gnssStatusCallback);
        isGnssStatusRegistered = false;
    }
}

/**
 * 统计 GNSS 卫星状态, 并根据卫星几何分布估算 DOP
 */
private void updateGnssStatus(GnssStatus status) {
    int visible = status.getSatelliteCount();
    int used = 0;
    int cn0Count = 0;
    int elevationCount = 0;
    double cn0Sum = 0.0;
    double cn0Max = 0.0;
    double elevationSum = 0.0;

    int gps = 0;
    int beidou = 0;
    int glonass = 0;
    int galileo = 0;

    ArrayList<double[]> usedRows = new ArrayList<>();
    ArrayList<double[]> allRows = new ArrayList<>();

    for (int i = 0; i < visible; i++) {
        boolean usedInFix = status.usedInFix(i);
        if (usedInFix) {
            used++;
        }

        double cn0 = status.getCn0DbHz(i);
        if (cn0 > 0) {
            cn0Sum += cn0;
            cn0Max = Math.max(cn0Max, cn0);
            cn0Count++;
        }

        double elevation = status.getElevationDegrees(i);
        double azimuth = status.getAzimuthDegrees(i);
        elevationSum += elevation;
        elevationCount++;

        double[] dopRow = buildDopRow(elevation, azimuth);
        allRows.add(dopRow);
        if (usedInFix) {

```

```

        usedRows.add(dopRow);
    }

    switch (status.getConstellationType(i)) {
        case GnssStatus.CONSTELLATION_GPS:
            gps++;
            break;
        case GnssStatus.CONSTELLATION_BEIDOU:
            beidou++;
            break;
        case GnssStatus.CONSTELLATION_GLOPASS:
            glonass++;
            break;
        case GnssStatus.CONSTELLATION_GALILEO:
            galileo++;
            break;
        default:
            break;
    }
}

gnssVisibleCount = visible;
gnssUsedCount = used;
gnssAvgCn0 = cn0Count == 0 ? 0.0 : cn0Sum / cn0Count;
gnssMaxCn0 = cn0Max;
gnssAvgElevation = elevationCount == 0 ? 0.0 : elevationSum / elevationCount;
gpsCount = gps;
beidouCount = beidou;
glonassCount = glonass;
galileoCount = galileo;

// DOP 需要至少 4 颗卫星。优先使用参与定位的卫星；不足 4 颗时退回全部可见卫星估算。
ArrayList<double[]> dopRows = usedRows.size() >= 4 ? usedRows : allRows;
calculateDop(dopRows);

gnssQualityLevel = evaluateGnssQualityLevel();

runOnUiThread(this::updateGnssQualityText);
}

/**
 * 根据卫星高度角、方位角构造 DOP 计算矩阵的一行
 */
private double[] buildDopRow(double elevationDeg, double azimuthDeg) {
    double elevation = Math.toRadians(elevationDeg);
    double azimuth = Math.toRadians(azimuthDeg);
    double cosEl = Math.cos(elevation);

    // 行向量包含东、北、天三个方向分量和接收机钟差项
    return new double[]{
        cosEl * Math.sin(azimuth),
        cosEl * Math.cos(azimuth),
        Math.sin(elevation),
        1.0
    };
}

/**
 * 由卫星几何矩阵估算 GDOP、PDOP、HDOP、VDOP
 */
private void calculateDop(ArrayList<double[]> rows) {
    if (rows.size() < 4) {
        gnssGdop = Double.NaN;
        gnssPdop = Double.NaN;
        gnssHdop = Double.NaN;
        gnssVdop = Double.NaN;
        return;
    }

    double[][] normal = new double[4][4];
    for (double[] row : rows) {
        for (int i = 0; i < 4; i++) {
            for (int j = 0; j < 4; j++) {
                normal[i][j] += row[i] * row[j];
            }
        }
    }

    double[][] inv = invert4x4(normal);
    if (inv == null) {
        gnssGdop = Double.NaN;
        gnssPdop = Double.NaN;
        gnssHdop = Double.NaN;
        gnssVdop = Double.NaN;
        return;
    }

    gnssGdop = safeSqrt(inv[0][0] + inv[1][1] + inv[2][2] + inv[3][3]);
    gnssPdop = safeSqrt(inv[0][0] + inv[1][1] + inv[2][2]);
    gnssHdop = safeSqrt(inv[0][0] + inv[1][1]);
    gnssVdop = safeSqrt(inv[2][2]);
}

/**
 * 4 阶矩阵求逆，采用高斯-约旦消元，避免引入额外数学库
 */
private double[][] invert4x4(double[][] matrix) {
    int n = 4;
    double[][] aug = new double[n][n * 2];

```

```

    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            aug[i][j] = matrix[i][j];
        }
        aug[i][i + n] = 1.0;
    }

    for (int col = 0; col < n; col++) {
        int pivot = col;
        for (int row = col + 1; row < n; row++) {
            if (Math.abs(aug[row][col]) > Math.abs(aug[pivot][col])) {
                pivot = row;
            }
        }

        if (Math.abs(aug[pivot][col]) < 1e-10) {
            return null;
        }

        if (pivot != col) {
            double[] temp = aug[col];
            aug[col] = aug[pivot];
            aug[pivot] = temp;
        }

        double pivotValue = aug[col][col];
        for (int j = 0; j < n * 2; j++) {
            aug[col][j] /= pivotValue;
        }

        for (int row = 0; row < n; row++) {
            if (row == col) {
                continue;
            }
            double factor = aug[row][col];
            for (int j = 0; j < n * 2; j++) {
                aug[row][j] -= factor * aug[col][j];
            }
        }
    }

    double[][] inv = new double[n][n];
    for (int i = 0; i < n; i++) {
        inv[i] = Arrays.copyOfRange(aug[i], n, n * 2);
    }
    return inv;
}

/**
 * 避免由于数值误差导致根号内出现很小的负数
 */
private double safeSqrt(double value) {
    if (Double.isNaN(value) || value < -1e-8) {
        return Double.NaN;
    }
    return Math.sqrt(Math.max(0.0, value));
}

/**
 * 综合可见卫星数、参与定位卫星数、C/N0 和 PDOP 给出 GNSS 定位质量等级
 */
private String evaluateGnssQualityLevel() {
    boolean pdopGood = Double.isNaN(gnssPdop) || gnssPdop <= 2.5;
    boolean pdopMiddle = Double.isNaN(gnssPdop) || gnssPdop <= 5.0;

    if (gnssUsedCount >= 8 && gnssAvgCn0 >= 35.0 && pdopGood) {
        return "优";
    } else if (gnssUsedCount >= 6 && gnssAvgCn0 >= 30.0 && pdopMiddle) {
        return "良";
    } else if (gnssUsedCount >= 4 && gnssAvgCn0 >= 20.0) {
        return "中";
    } else if (gnssVisibleCount > 0) {
        return "差";
    } else {
        return "等待卫星数据";
    }
}

/**
 * 刷新 GNSS 质量状态文字
 */
private void updateGnssQualityText() {
    if (tvGnssQuality == null) {
        return;
    }

    if (gnssVisibleCount == 0) {
        tvGnssQuality.setText("GNSS 质量: 等待卫星数据");
        updateQuickStatusCards(null);
        if ("gnss".equals(activeFeaturePanel)) {
            refreshFeaturePanel();
        }
        return;
    }

    tvGnssQuality.setText(String.format(
        Locale.getDefault(),
        "GNSS 质量: %s | 可见%d颗/参与%d颗 | 平均 C/N0 %.1fdB-Hz | PDOP%s HDOP%s VDOP%s\n 星座: GPS%d 北斗%d GLONASS%d

```

```

Galileo%d | 平均高度角%.1f°",
        gnssQualityLevel,
        gnssVisibleCount,
        gnssUsedCount,
        gnssAvgCn0,
        formatDop(gnssPdop),
        formatDop(gnssHdop),
        formatDop(gnssVdop),
        gpsCount,
        beidouCount,
        glonassCount,
        galileoCount,
        gnssAvgElevation
    ));
    updateQuickStatusCards(null);
    if ("gnss".equals(activeFeaturePanel)) {
        refreshFeaturePanel();
    }
}

/**
 * DOP 显示格式化
 */
private String formatDop(double value) {
    if (Double.isNaN(value) || Double.isInfinite(value)) {
        return "--";
    }
    return String.format(Locale.getDefault(), "%.1f", value);
}

// ===== 定位精度评价、误差源分析、异常检测、组合导航 =====

/**
 * 初始化加速度计和磁力计，用于给出手机姿态航向，辅助 GNSS 航向判断
 */
private void initSensorFusion() {
    sensorManager = (SensorManager) getSystemService(Context.SENSOR_SERVICE);
    if (sensorManager != null) {
        accelerometerSensor = sensorManager.getDefaultSensor(Sensor.TYPE_ACCELEROMETER);
        magneticSensor = sensorManager.getDefaultSensor(Sensor.TYPE_MAGNETIC_FIELD);
    }
    updateSensorFusionText();
}

/**
 * 页面显示时注册传感器监听
 */
private void startSensorMonitor() {
    if (sensorManager == null) {
        return;
    }
    if (accelerometerSensor != null) {
        sensorManager.registerListener(this, accelerometerSensor, SensorManager.SENSOR_DELAY_UI);
    }
    if (magneticSensor != null) {
        sensorManager.registerListener(this, magneticSensor, SensorManager.SENSOR_DELAY_UI);
    }
}

/**
 * 页面暂停或销毁时停止传感器监听，避免资源占用
 */
private void stopSensorMonitor() {
    if (sensorManager != null) {
        sensorManager.unregisterListener(this);
    }
}

/**
 * 传感器数据刷新回调，用加速度计和磁力计估计手机朝向
 */
@Override
public void onSensorChanged(SensorEvent event) {
    if (event == null || event.sensor == null) {
        return;
    }

    if (event.sensor.getType() == Sensor.TYPE_ACCELEROMETER) {
        lowPass(event.values, accelerometerValues);
        hasAccelerometerValue = true;
    } else if (event.sensor.getType() == Sensor.TYPE_MAGNETIC_FIELD) {
        lowPass(event.values, magneticValues);
        hasMagneticValue = true;
    }

    if (hasAccelerometerValue && hasMagneticValue) {
        float[] rotationMatrix = new float[9];
        float[] orientation = new float[3];
        boolean success = SensorManager.getRotationMatrix(
            rotationMatrix,
            null,
            accelerometerValues,
            magneticValues
        );
        if (success) {
            SensorManager.getOrientation(rotationMatrix, orientation);
            float azimuth = (float) Math.toDegrees(orientation[0]);
            sensorHeadingDeg = normalizeAngle(azimuth);
        }
    }
}

```

```

        updateSensorFusionText();
    }
}

/**
 * 传感器精度变化回调，当前仅保留接口实现
 */
@Override
public void onAccuracyChanged(Sensor sensor, int accuracy) {
    // 当前功能只用于课堂展示辅助航向，不单独处理传感器精度等级
}

/**
 * 对传感器数据做简单低通滤波，减少航向文字跳动
 */
private void lowPass(float[] input, float[] output) {
    final float alpha = 0.15f;
    for (int i = 0; i < output.length && i < input.length; i++) {
        output[i] = output[i] + alpha * (input[i] - output[i]);
    }
}

/**
 * 定位成功后统一刷新新增的任务二扩展功能
 */
private void updateExtendedGnssFunctions(AMapLocation location) {
    if (location == null) {
        return;
    }

    currentGnssSpeed = Math.max(0f, location.getSpeed());
    currentGnssBearing = normalizeAngle(location.getBearing());

    updateAccuracyEvalText(location);
    updateErrorAnalysisText(location);
    updateAnomalyState(location);
    updateAnomalyDetectText();
    updateSensorFusionText();
}

/**
 * 显示定位精度评价：系统水平精度、定位方式、速度、航向和使用建议
 */
private void updateAccuracyEvalText(AMapLocation location) {
    if (tvAccuracyEval == null) {
        return;
    }
    if (location == null) {
        tvAccuracyEval.setText("定位精度评价：等待定位数据");
        updateQuickStatusCards(null);
        if ("accuracy".equals(activeFeaturePanel)) {
            refreshFeaturePanel();
        }
        return;
    }

    float accuracy = location.getAccuracy();
    String level = evaluateAccuracyLevel(accuracy);
    String typeName = getLocationTypeName(location.getLocationType());
    String suggestion;
    if (accuracy <= 5f) {
        suggestion = "可用于精细导航/轨迹记录";
    } else if (accuracy <= 15f) {
        suggestion = "可用于普通导航";
    } else if (accuracy <= 30f) {
        suggestion = "适合粗略定位";
    } else {
        suggestion = "建议移动到开阔区域";
    }

    tvAccuracyEval.setText(String.format(
        Locale.getDefault(),
        "定位精度评价：%s | 水平精度%.1fm | 定位方式%s | 速度%.1fm/s | 航向%.0f° | %s",
        level,
        accuracy,
        typeName,
        Math.max(0f, location.getSpeed()),
        normalizeAngle(location.getBearing()),
        suggestion
    ));
    updateQuickStatusCards(location);
    if ("accuracy".equals(activeFeaturePanel)) {
        refreshFeaturePanel();
    }
}

/**
 * 根据 GNSS 信号、卫星几何和定位方式推断可能的误差来源
 */
private void updateErrorAnalysisText(AMapLocation location) {
    if (tvErrorAnalysis == null) {
        return;
    }
    if (location == null) {
        tvErrorAnalysis.setText("误差源分析：等待定位数据");
        if ("error".equals(activeFeaturePanel)) {
            refreshFeaturePanel();
        }
    }
}

```

```

        }
        return;
    }

    ArrayList<String> reasons = new ArrayList<>();
    float accuracy = location.getAccuracy();
    int locationType = location.getLocationType();

    if (accuracy > 30f) {
        reasons.add("水平精度较差");
    }
    if (gnssVisibleCount == 0) {
        reasons.add("暂未获取卫星状态, 可能为网络/融合定位");
    } else {
        if (gnssUsedCount > 0 && gnssUsedCount < 4) {
            reasons.add("参与解算卫星不足");
        }
        if (gnssAvgCn0 > 0 && gnssAvgCn0 < 25.0) {
            reasons.add("C/N0 偏低, 疑似遮挡或信号弱");
        }
        if (!Double.isNaN(gnssPdop) && gnssPdop > 5.0) {
            reasons.add("PDOP 偏大, 卫星几何构型较差");
        }
        if (gnssAvgElevation > 0 && gnssAvgElevation < 25.0) {
            reasons.add("平均高度角低, 易受多路径影响");
        }
    }

    if (locationType != 1) {
        reasons.add("定位方式非纯 GPS, 可能受 WiFi/基站环境影响");
    }
    if (reasons.isEmpty()) {
        reasons.add("卫星信号和几何条件较正常");
    }

    tvErrorAnalysis.setText("误差源分析: " + joinReasons(reasons));
    if ("error".equals(activeFeaturePanel)) {
        refreshFeaturePanel();
    }
}

/**
 * 更新异常/欺骗初筛状态: 位置跳变、速度异常、卫星状态与定位精度矛盾
 */
private void updateAnomalyState(AMapLocation location) {
    LatLng now = new LatLng(location.getLatitude(), location.getLongitude());
    long nowTime = System.currentTimeMillis();
    float accuracy = location.getAccuracy();
    ArrayList<String> reasons = new ArrayList<>();

    if (lastAnomalyLatLng != null && lastAnomalyTimeMillis > 0L) {
        double dt = (nowTime - lastAnomalyTimeMillis) / 1000.0;
        if (dt > 0.2) {
            lastJumpDistance = AMapUtils.calculateLineDistance(lastAnomalyLatLng, now);
            lastJumpSpeed = lastJumpDistance / dt;

            if (lastJumpDistance > Math.max(50.0, accuracy * 3.0) && lastJumpSpeed > 35.0) {
                reasons.add("短时间位置跳变明显");
            }
            if (lastJumpSpeed > 80.0 && location.getSpeed() < 15.0) {
                reasons.add("坐标推算速度与系统速度不一致");
            }
        }
    }

    if (gnssVisibleCount > 0 && gnssUsedCount < 4 && accuracy <= 10f) {
        reasons.add("卫星不足但系统精度很高, 需复核定位来源");
    }
    if (gnssAvgCn0 > 0 && gnssAvgCn0 < 18.0 && accuracy <= 10f) {
        reasons.add("弱信号下仍给出高精度, 需警惕异常定位");
    }
    if (!Float.isNaN(sensorHeadingDeg) && location.getSpeed() > 2.0f) {
        double headingDiff = angleDifference(sensorHeadingDeg, normalizeAngle(location.getBearing()));
        if (headingDiff > 120.0) {
            reasons.add("GNSS 航向与手机姿态航向差异过大");
        }
    }

    if (reasons.isEmpty()) {
        anomalyLevel = "正常";
        anomalyReason = String.format(
            Locale.getDefault(),
            "未发现明显跳变, 最近跳变%.1fm/%.1fm/s",
            lastJumpDistance,
            lastJumpSpeed
        );
    } else if (reasons.size() == 1) {
        anomalyLevel = "轻微异常";
        anomalyReason = joinReasons(reasons);
    } else {
        anomalyLevel = "疑似异常";
        anomalyReason = joinReasons(reasons);
    }

    lastAnomalyLatLng = now;
    lastAnomalyTimeMillis = nowTime;
}

```

```

/**
 * 显示异常/欺骗初筛结果
 */
private void updateAnomalyDetectText() {
    if (tvAnomalyDetect == null) {
        return;
    }
    tvAnomalyDetect.setText("异常/欺骗初筛: " + anomalyLevel + " | " + anomalyReason);
    updateQuickStatusCards(null);
    if ("anomaly".equals(activeFeaturePanel)) {
        refreshFeaturePanel();
    }
}

/**
 * 显示 GNSS + 惯性传感器辅助导航状态
 */
private void updateSensorFusionText() {
    if (tvSensorFusion == null) {
        return;
    }

    if (accelerometerSensor == null || magneticSensor == null) {
        tvSensorFusion.setText("组合导航: 本机缺少加速度计或磁力计, 无法显示惯性辅助航向");
        if ("sensor".equals(activeFeaturePanel)) {
            refreshFeaturePanel();
        }
        return;
    }

    if (Float.isNaN(sensorHeadingDeg)) {
        tvSensorFusion.setText("组合导航: 等待传感器航向数据");
        if ("sensor".equals(activeFeaturePanel)) {
            refreshFeaturePanel();
        }
        return;
    }

    String state;
    if (currentLatLng == null) {
        state = "等待 GNSS 定位";
    } else if (currentGnssSpeed < 0.8f) {
        state = "低速/静止, 航向以手机传感器为参考";
    } else {
        double diff = angleDifference(sensorHeadingDeg, currentGnssBearing);
        if (diff <= 45.0) {
            state = String.format(Locale.getDefault(), "航向一致, 差值%.0f°", diff);
        } else {
            state = String.format(Locale.getDefault(), "航向差异%.0f°, 可能处于转弯或磁干扰环境", diff);
        }
    }

    tvSensorFusion.setText(String.format(
        Locale.getDefault(),
        "组合导航: 传感器航向%.0f° | GNSS 速度%.1fm/s | GNSS 航向%s | %s",
        sensorHeadingDeg,
        currentGnssSpeed,
        Float.isNaN(currentGnssBearing) ? "--" : String.format(Locale.getDefault(), "%.0f°", currentGnssBearing),
        state
    ));
    if ("sensor".equals(activeFeaturePanel)) {
        refreshFeaturePanel();
    }
}

/**
 * 角度归一化到 0~360 度
 */
private float normalizeAngle(float angle) {
    if (Float.isNaN(angle) || Float.isInfinite(angle)) {
        return Float.NaN;
    }
    float normalized = angle % 360f;
    if (normalized < 0) {
        normalized += 360f;
    }
    return normalized;
}

/**
 * 两个方向角的最小夹角差, 范围 0~180 度
 */
private double angleDifference(double a, double b) {
    if (Double.isNaN(a) || Double.isNaN(b)) {
        return 0.0;
    }
    double diff = Math.abs(a - b) % 360.0;
    return diff > 180.0 ? 360.0 - diff : diff;
}

/**
 * 将若干原因拼接成适合界面展示的文字
 */
private String joinReasons(ArrayList<String> reasons) {
    StringBuilder builder = new StringBuilder();
    for (int i = 0; i < reasons.size(); i++) {
        if (i > 0) {
            builder.append(": ");
        }
    }
}

```

```

        builder.append(reasons.get(i));
    }
    return builder.toString();
}

/**
 * 高德定位类型转为可读文字，避免界面只显示数字
 */
private String getLocationTypeName(int type) {
    switch (type) {
        case 1:
            return "GPS";
        case 2:
            return "前次定位";
        case 4:
            return "缓存定位";
        case 5:
            return "WiFi 定位";
        case 6:
            return "基站定位";
        case 8:
            return "离线定位";
        case 9:
            return "末次缓存";
        default:
            return "融合定位" + type;
    }
}

// ===== 围栏 =====

/**
 * 开关围栏模式
 */
private void toggleFenceMode() {
    isFenceModeEnabled = !isFenceModeEnabled;

    if (isFenceModeEnabled) {
        btnFenceMode.setText("围栏中");
        cardFenceRadius.setVisibility(View.VISIBLE);
        Toast.makeText(this, "围栏模式已开启，请长按地图设置围栏中心", Toast.LENGTH_SHORT).show();
    } else {
        btnFenceMode.setText("围栏");
        if (fenceCenter == null) {
            cardFenceRadius.setVisibility(View.GONE);
        }
        Toast.makeText(this, "围栏模式已关闭", Toast.LENGTH_SHORT).show();
    }
}

/**
 * 地图长按事件
 * 在围栏模式下，长按地图设置围栏中心
 */
@Override
public void onMapLongClick(LatLng latLng) {
    if (!isFenceModeEnabled) {
        return;
    }

    // 记录新的围栏中心
    fenceCenter = latLng;
    // 重置围栏内外状态
    lastInsideState = null;

    // 如果旧围栏存在则先移除
    if (fenceCircle != null) {
        fenceCircle.remove();
    }
    if (fenceMarker != null) {
        fenceMarker.remove();
    }

    // 添加围栏中心点
    fenceMarker = aMap.addMarker(new MarkerOptions()
        .position(fenceCenter)
        .title("电子围栏中心")
        .icon(BitmapDescriptorFactory.defaultMarker(BitmapDescriptorFactory.HUE_AZURE)));

    // 添加围栏圆形区域
    fenceCircle = aMap.addCircle(new CircleOptions()
        .center(fenceCenter)
        .radius(fenceRadius)
        .strokeWidth(4)
        .strokeColor(0xAA64748B)
        .fillColor(0x2264748B));

    cardFenceRadius.setVisibility(View.VISIBLE);
    Toast.makeText(this, "电子围栏已设置", Toast.LENGTH_SHORT).show();

    // 如果已有当前位置，则立刻更新围栏状态
    if (currentLatLng != null) {
        checkFenceState(currentLatLng, false);
    }
}

```

```

/**
 * 判断某个位置点是否在围栏内
 * allowToast 为 true 时允许弹出进入或离开提示
 */
private void checkFenceState(LatLng locationLatLng, boolean allowToast) {
    if (fenceCenter == null) {
        updateFenceStatusNoFence();
        return;
    }

    // 计算当前位置到围栏中心的距离
    float distance = AMapUtils.calculateLineDistance(locationLatLng, fenceCenter);
    // 小于等于围栏半径则认为在围栏内
    boolean isInside = distance <= fenceRadius;

    // 组织状态显示文本
    String statusText = String.format(
        Locale.getDefault(),
        "围栏: %s | 距中心 %.1fm | 半径 %.0fm",
        isInside ? "围栏内" : "围栏外",
        distance,
        fenceRadius
    );

    tvFenceStatus.setText(statusText);
    tvFenceStatus.setTextColor(isInside ? 0xFF475569 : 0xFF78716C);

    // 第一次判断时仅记录状态, 不弹进入或离开提示
    if (lastInsideState == null) {
        lastInsideState = isInside;
        return;
    }

    // 仅在状态发生变化时处理提示
    if (isInside != lastInsideState) {
        if (allowToast) {
            if (isInside) {
                Toast.makeText(this, "已进入电子围栏", Toast.LENGTH_SHORT).show();
            } else {
                Toast.makeText(this, "已离开电子围栏", Toast.LENGTH_SHORT).show();
            }
        }
        lastInsideState = isInside;
    }
}

/**
 * 显示“围栏未设置”的默认状态
 */
private void updateFenceStatusNoFence() {
    tvFenceStatus.setText("围栏: 未设置");
    tvFenceStatus.setTextColor(0xFF64748B);
}

/**
 * 清除当前围栏
 */
private void clearFence() {
    if (fenceCircle != null) {
        fenceCircle.remove();
        fenceCircle = null;
    }
    if (fenceMarker != null) {
        fenceMarker.remove();
        fenceMarker = null;
    }

    fenceCenter = null;
    lastInsideState = null;
    cardFenceRadius.setVisibility(View.GONE);
    btnFenceMode.setText("围栏");
    isFenceModeEnabled = false;
    updateFenceStatusNoFence();

    Toast.makeText(this, "围栏已清除", Toast.LENGTH_SHORT).show();
}

// ===== 轨迹 =====

/**
 * 开始或停止轨迹记录
 */
private void toggleTrackRecording() {
    isTracking = !isTracking;

    if (isTracking) {
        btnTrack.setText("停轨");
        Toast.makeText(this, "开始记录轨迹", Toast.LENGTH_SHORT).show();
    } else {
        btnTrack.setText("轨迹");
        Toast.makeText(this, "已停止记录轨迹", Toast.LENGTH_SHORT).show();
    }

    updateTrackStatus();
}

```

```

/**
 * 向轨迹中加入一个新点
 * 只有与上一个点距离至少达到 2 米时才真正加入
 */
private void addTrackPoint(LatLng point) {
    if (trackPoints.isEmpty()) {
        trackPoints.add(point);
    } else {
        LatLng lastPoint = trackPoints.get(trackPoints.size() - 1);
        float distance = AMapUtils.calculateLineDistance(lastPoint, point);

        if (distance >= 2f) {
            trackDistance += distance;
            trackPoints.add(point);
        }
    }

    // 首次绘制轨迹时创建折线, 否则更新点集合
    if (trackPolyline == null) {
        trackPolyline = aMap.addPolyline(new PolylineOptions()
            .addAll(trackPoints)
            .width(10f)
            .color(0xFF64748B));
    } else {
        trackPolyline.setPoints(new ArrayList<>(trackPoints));
    }

    updateTrackStatus();
}

/**
 * 清除轨迹
 */
private void clearTrack() {
    isTracking = false;
    btnTrack.setText("轨迹");

    trackPoints.clear();
    trackDistance = 0f;

    if (trackPolyline != null) {
        trackPolyline.remove();
        trackPolyline = null;
    }

    updateTrackStatus();
    Toast.makeText(this, "轨迹已清除", Toast.LENGTH_SHORT).show();
}

/**
 * 更新顶部轨迹状态文字
 */
private void updateTrackStatus() {
    if (trackPoints.isEmpty()) {
        tvTrackStatus.setText("轨迹: 未记录");
        return;
    }

    if (isTracking) {
        tvTrackStatus.setText(String.format(
            Locale.getDefault(),
            "轨迹: 记录中 | 点数 %d | 距离 %.1fm",
            trackPoints.size(),
            trackDistance
        ));
    } else {
        tvTrackStatus.setText(String.format(
            Locale.getDefault(),
            "轨迹: 已停止 | 点数 %d | 总长 %.1fm",
            trackPoints.size(),
            trackDistance
        ));
    }
}

// ===== 搜索与路线 =====

/**
 * 根据输入框关键字搜索地点
 */
private void searchLocation() {
    String keyword = etKeyword.getText().toString().trim();
    if (keyword.isEmpty()) {
        Toast.makeText(this, "请输入要搜索的位置", Toast.LENGTH_SHORT).show();
        return;
    }

    // 搜索时确保顶部信息区展开
    isInfoExpanded = true;
    layoutSearchExtras.setVisibility(View.VISIBLE);
    btnToggleInfo.setText("收起");

    // 构造 POI 查询对象
    PoiSearch.Query query = new PoiSearch.Query(keyword, "", "");
    query.setPageSize(20);
    query.setPageNum(1);

    try {

```

```

        PoiSearch poiSearch = new PoiSearch(this, query);
        poiSearch.setOnPoiSearchListener(this);
        poiSearch.searchPOIAsyn();
        tvRouteStatus.setText("路线: 正在搜索位置...");
    } catch (AMapException e) {
        e.printStackTrace();
        tvRouteStatus.setText("路线: 位置搜索初始化失败");
        Toast.makeText(this, "位置搜索初始化失败", Toast.LENGTH_SHORT).show();
    }
}

/**
 * POI 搜索结果回调
 */
@Override
public void onPoiSearched(PoiResult pageResult, int errorCode) {
    searchPoiList.clear();

    if (errorCode == 1000 && pageResult != null && pageResult.getPois() != null
        && !pageResult.getPois().isEmpty()) {

        searchPoiList.addAll(pageResult.getPois());
        searchAdapter.notifyDataSetChanged();

        lvSearchResults.setVisibility(View.VISIBLE);
        tvRouteStatus.setText("路线: 请选择一个搜索结果");
    } else {
        lvSearchResults.setVisibility(View.GONE);
        layoutRouteActions.setVisibility(View.GONE);
        tvRouteStatus.setText("路线: 未搜索到结果");
        Toast.makeText(this, "未搜索到结果", Toast.LENGTH_SHORT).show();
    }
}

/**
 * 点击某条搜索结果后, 设置目的地并在地图上添加目的地 Marker
 */
private void selectSearchResult(int position) {
    if (position < 0 || position >= searchPoiList.size()) {
        return;
    }

    PoiItem item = searchPoiList.get(position);
    LatLonPoint point = item.getLatLonPoint();

    if (point == null) {
        Toast.makeText(this, "该结果没有有效坐标", Toast.LENGTH_SHORT).show();
        return;
    }

    routeDestination = new LatLng(point.getLatitude(), point.getLongitude());
    routeDestinationName = item.getTitle() == null ? "目的地" : item.getTitle();

    if (destinationMarker != null) {
        destinationMarker.remove();
    }

    destinationMarker = aMap.addMarker(new MarkerOptions()
        .position(routeDestination)
        .title(routeDestinationName)
        .snippet(item.getSnippet())
        .icon(BitmapDescriptorFactory.defaultMarker(BitmapDescriptorFactory.HUE_ORANGE)));

    aMap.animateCamera(CameraUpdateFactory.newLatLngZoom(routeDestination, 16f));
    lvSearchResults.setVisibility(View.GONE);

    layoutRouteActions.setVisibility(View.VISIBLE);
    tvSelectedDestination.setText("已选目的地: " + routeDestinationName);
    tvRouteStatus.setText("路线: 已选目的地, 未规划");
    Toast.makeText(this, "已选择: " + routeDestinationName, Toast.LENGTH_SHORT).show();
}

/**
 * 单个 POI 搜索回调
 * 当前代码没有使用
 */
@Override
public void onPoiItemSearched(PoiItem poiItem, int errorCode) {
    // 未使用
}

/**
 * 规划步行路线
 */
private void planWalkRoute() {
    if (currentLatLng == null) {
        Toast.makeText(this, "当前位置未获取到", Toast.LENGTH_SHORT).show();
        return;
    }

    if (routeDestination == null) {
        Toast.makeText(this, "请先搜索并选择一个目的地", Toast.LENGTH_SHORT).show();
        return;
    }

    if (routeSearch == null) {
        Toast.makeText(this, "路线服务初始化失败", Toast.LENGTH_SHORT).show();
    }
}

```

```

        return;
    }

    RouteSearch.FromAndTo fromAndTo = new RouteSearch.FromAndTo(
        new LatLonPoint(currentLatLng.latitude, currentLatLng.longitude),
        new LatLonPoint(routeDestination.latitude, routeDestination.longitude)
    );

    RouteSearch.WalkRouteQuery query = new RouteSearch.WalkRouteQuery(fromAndTo);
    routeSearch.calculateWalkRouteAsync(query);

    tvRouteStatus.setText("路线：正在规划步行路线...");
}

/**
 * 规划驾车路线
 */
private void planDriveRoute() {
    if (currentLatLng == null) {
        Toast.makeText(this, "当前位置未获取到", Toast.LENGTH_SHORT).show();
        return;
    }

    if (routeDestination == null) {
        Toast.makeText(this, "请先搜索并选择一个目的地", Toast.LENGTH_SHORT).show();
        return;
    }

    if (routeSearch == null) {
        Toast.makeText(this, "路线服务初始化失败", Toast.LENGTH_SHORT).show();
        return;
    }

    RouteSearch.FromAndTo fromAndTo = new RouteSearch.FromAndTo(
        new LatLonPoint(currentLatLng.latitude, currentLatLng.longitude),
        new LatLonPoint(routeDestination.latitude, routeDestination.longitude)
    );

    RouteSearch.DriveRouteQuery query = new RouteSearch.DriveRouteQuery(
        fromAndTo,
        RouteSearch.DRIVING_SINGLE_DEFAULT,
        null,
        null,
        ""
    );

    routeSearch.calculateDriveRouteAsync(query);
    tvRouteStatus.setText("路线：正在规划驾车路线...");
}

/**
 * 步行路线结果回调
 */
@Override
public void onWalkRouteSearched(WalkRouteResult walkRouteResult, int errorCode) {
    if (errorCode == 1000
        && walkRouteResult != null
        && walkRouteResult.getPaths() != null
        && !walkRouteResult.getPaths().isEmpty()) {

        WalkPath walkPath = walkRouteResult.getPaths().get(0);
        List<LatLng> latLngs = new ArrayList<>();

        if (currentLatLng != null) {
            latLngs.add(currentLatLng);
        }

        // 把每一步的折线点拼接成整条路线
        for (WalkStep step : walkPath.getSteps()) {
            List<LatLonPoint> points = step.getPolyline();
            if (points != null) {
                for (LatLonPoint p : points) {
                    latLngs.add(new LatLng(p.getLatitude(), p.getLongitude()));
                }
            }
        }

        drawRoutePolyline(latLngs, 0xFF1D4ED8);

        tvRouteStatus.setText(String.format(
            Locale.getDefault(),
            "路线：步行 %.0fm | 约 %.0f 分钟",
            walkPath.getDistance(),
            walkPath.getDuration() / 60f
        ));

        Toast.makeText(this, "步行路线规划成功", Toast.LENGTH_SHORT).show();
    } else {
        tvRouteStatus.setText("路线：步行路线规划失败");
        Toast.makeText(this, "步行路线规划失败", Toast.LENGTH_SHORT).show();
    }
}

/**
 * 驾车路线结果回调
 */
@Override

```

```

public void onDriveRouteSearched(DriveRouteResult driveRouteResult, int errorCode) {
    if (errorCode == 1000
        && driveRouteResult != null
        && driveRouteResult.getPaths() != null
        && !driveRouteResult.getPaths().isEmpty()) {

        DrivePath drivePath = driveRouteResult.getPaths().get(0);
        List<LatLng> latLngs = new ArrayList<>();

        if (currentLatLng != null) {
            latLngs.add(currentLatLng);
        }

        // 把每一步驾车路线点拼接成整条路线
        for (com.amap.api.services.route.DriveStep step : drivePath.getSteps()) {
            List<LatLngPoint> points = step.getPolyline();
            if (points != null) {
                for (LatLngPoint p : points) {
                    latLngs.add(new LatLng(p.getLatitude(), p.getLongitude()));
                }
            }
        }

        drawRoutePolyline(latLngs, 0xFFFF6B00);

        tvRouteStatus.setText(String.format(
            Locale.getDefault(),
            "路线: 驾车 %.0fm | 约 %.0f 分钟",
            drivePath.getDistance(),
            drivePath.getDuration() / 60f
        ));

        Toast.makeText(this, "驾车路线规划成功", Toast.LENGTH_SHORT).show();
    } else {
        tvRouteStatus.setText("路线: 驾车路线规划失败");
        Toast.makeText(this, "驾车路线规划失败", Toast.LENGTH_SHORT).show();
    }
}

/**
 * 绘制路线折线, 并自动缩放地图让整条路线完整显示
 */
private void drawRoutePolyline(List<LatLng> latLngs, int color) {
    if (routePolyline != null) {
        routePolyline.remove();
    }

    routePolyline = aMap.addPolyline(new PolylineOptions()
        .addAll(latLngs)
        .width(14f)
        .color(color));

    if (!latLngs.isEmpty()) {
        LatLngBounds.Builder builder = new LatLngBounds.Builder();
        for (LatLng latLng : latLngs) {
            builder.include(latLng);
        }
        aMap.animateCamera(CameraUpdateFactory.newLatLngBounds(builder.build(), 120));
    }
}

/**
 * 公交路线回调
 * 当前未使用
 */
@Override
public void onBusRouteSearched(BusRouteResult busRouteResult, int errorCode) {
    // 未使用
}

/**
 * 骑行路线回调
 * 当前未使用
 */
@Override
public void onRideRouteSearched(RideRouteResult rideRouteResult, int errorCode) {
    // 未使用
}

/**
 * 清除当前路线
 */
private void clearRoute() {
    if (routePolyline != null) {
        routePolyline.remove();
        routePolyline = null;
    }
    tvRouteStatus.setText("路线: 已清除");
    Toast.makeText(this, "路线已清除", Toast.LENGTH_SHORT).show();
}

// ===== 拍照 =====

/**
 * 在当前位置调用相机拍照, 并准备后续生成拍照定位点
 */
private void takePhotoWithLocation() {

```

```

        if (currentLatLng == null) {
            Toast.makeText(this, "当前位置未获取到, 稍后再试", Toast.LENGTH_SHORT).show();
            return;
        }

        if (ContextCompat.checkSelfPermission(this, Manifest.permission.CAMERA)
            != PackageManager.PERMISSION_GRANTED) {
            ActivityCompat.requestPermissions(
                this,
                new String[]{Manifest.permission.CAMERA},
                REQUEST_CAMERA_PERMISSION
            );
            return;
        }

        Intent takePictureIntent = new Intent(MediaStore.ACTION_IMAGE_CAPTURE);
        if (takePictureIntent.resolveActivity(getPackageManager()) != null) {
            // 先记录拍照发生时的位置、时间、地址
            pendingPhotoLatLng = currentLatLng;
            pendingPhotoTime = timeFormat.format(new Date());
            pendingPhotoAddress = currentAddress;
            startActivityForResult(takePictureIntent, REQUEST_IMAGE_CAPTURE);
        } else {
            Toast.makeText(this, "未找到可用相机应用", Toast.LENGTH_SHORT).show();
        }
    }

    /**
     * 接收系统相机返回结果
     * 成功后在地图上创建拍照定位点
     */
    @Override
    protected void onActivityResult(int requestCode, int resultCode, Intent data) {
        super.onActivityResult(requestCode, resultCode, data);

        if (requestCode == REQUEST_IMAGE_CAPTURE && resultCode == RESULT_OK && data != null) {
            Bundle extras = data.getExtras();
            if (extras != null && extras.get("data") instanceof Bitmap) {
                Bitmap bitmap = (Bitmap) extras.get("data");

                if (pendingPhotoLatLng != null) {
                    Marker marker = aMap.addMarker(new MarkerOptions()
                        .position(pendingPhotoLatLng)
                        .title("拍照定位点")
                        .snippet("点击查看照片")
                        .icon(BitmapDescriptorFactory.defaultMarker(BitmapDescriptorFactory.HUE_ROSE)));

                    if (marker != null) {
                        String info = "时间: " + pendingPhotoTime
                            + "\n 地址: " + pendingPhotoAddress
                            + "\n 经度: " + pendingPhotoLatLng.longitude
                            + "\n 纬度: " + pendingPhotoLatLng.latitude;

                        // 保存 Marker id 与照片、说明信息的对应关系
                        photoBitmapMap.put(marker.getId(), bitmap);
                        photoInfoMap.put(marker.getId(), info);
                        photoMarkers.add(marker);

                        // 把地图移动到拍照点位置
                        aMap.animateCamera(CameraUpdateFactory.newLatLngZoom(pendingPhotoLatLng, 18f));
                        Toast.makeText(this, "拍照定位成功", Toast.LENGTH_SHORT).show();
                    }
                }
            }
        }
    }

    /**
     * 清除所有拍照定位点
     */
    private void clearPhotoMarkers() {
        for (Marker marker : photoMarkers) {
            if (marker != null) {
                marker.remove();
            }
        }
        photoMarkers.clear();
        photoBitmapMap.clear();
        photoInfoMap.clear();
        Toast.makeText(this, "拍照点已清除", Toast.LENGTH_SHORT).show();
    }

    /**
     * Marker 点击事件
     * 如果点击的是拍照点, 就弹出照片信息对话框
     */
    @Override
    public boolean onMarkerClick(Marker marker) {
        if (photoBitmapMap.containsKey(marker.getId())) {
            showPhotoDialog(photoBitmapMap.get(marker.getId()), photoInfoMap.get(marker.getId()));
            return true;
        }
        return false;
    }

    /**
     * 弹出拍照定位信息对话框

```

```

*/
private void showPhotoDialog(Bitmap bitmap, String info) {
    LinearLayout layout = new LinearLayout(this);
    layout.setOrientation(LinearLayout.VERTICAL);
    layout.setPadding(dp(20), dp(20), dp(20), dp(10));

    ImageView imageView = new ImageView(this);
    imageView.setImageBitmap(bitmap);
    imageView.setAdjustViewBounds(true);

    TextView textView = new TextView(this);
    textView.setText(info);
    textView.setTextSize(15f);
    textView.setPadding(0, dp(16), 0, 0);

    layout.addView(imageView);
    layout.addView(textView);

    new AlertDialog.Builder(this)
        .setTitle("拍照定位信息")
        .setView(layout)
        .setPositiveButton("确定", null)
        .show();
}

// ===== 生命周期 =====

/**
 * 页面恢复显示时同步恢复 MapView
 */
@Override
protected void onResume() {
    super.onResume();
    mapView.onResume();
    startSensorMonitor();
}

/**
 * 页面暂停时同步暂停 MapView
 */
@Override
protected void onPause() {
    super.onPause();
    mapView.onPause();
    stopSensorMonitor();
}

/**
 * 保存页面状态时同步保存 MapView 状态
 */
@Override
protected void onSaveInstanceState(@NonNull Bundle outState) {
    super.onSaveInstanceState(outState);
    mapView.onSaveInstanceState(outState);
}

/**
 * 页面销毁时释放定位和地图资源
 */
@Override
protected void onDestroy() {
    super.onDestroy();
    if (aMapLocationClient != null) {
        aMapLocationClient.stopLocation();
        aMapLocationClient.onDestroy();
    }
    stopGnssStatusMonitor();
    stopSensorMonitor();
    mapView.onDestroy();
}

// ===== 权限回调 =====

/**
 * 动态权限申请结果回调
 */
@Override
public void onRequestPermissionsResult(int requestCode, @NonNull String[] permissions,
    @NonNull int[] grantResults) {
    super.onRequestPermissionsResult(requestCode, permissions, grantResults);

    // 处理定位权限结果
    if (requestCode == REQUEST_LOCATION_PERMISSION) {
        boolean granted = true;
        for (int result : grantResults) {
            if (result != PackageManager.PERMISSION_GRANTED) {
                granted = false;
                break;
            }
        }
        if (granted) {
            enableLocation();
        } else {
            Toast.makeText(this, "未授予定位权限", Toast.LENGTH_SHORT).show();
        }
    }
}

```

```

// 处理相机权限结果
if (requestCode == REQUEST_CAMERA_PERMISSION) {
    if (grantResults.length > 0 && grantResults[0] == PackageManager.PERMISSION_GRANTED) {
        takePhotoWithLocation();
    } else {
        Toast.makeText(this, "未授予相机权限", Toast.LENGTH_SHORT).show();
    }
}
}

// ===== 工具方法 =====

/**
 * dp 转 px
 * 方便在代码中动态创建控件时统一设置尺寸
 */
private int dp(int value) {
    return (int) TypedValue.applyDimension(
        TypedValue.COMPLEX_UNIT_DIP,
        value,
        getResources().getDisplayMetrics()
    );
}

// ===== 自定义搜索结果适配器 =====

/**
 * 搜索结果适配器
 * 不额外新建 XML 布局文件，而是在代码里动态生成列表项视图
 */
private class SearchResultAdapter extends BaseAdapter {

    @Override
    public int getCount() {
        return searchPoiList.size();
    }

    @Override
    public Object getItem(int position) {
        return searchPoiList.get(position);
    }

    @Override
    public long getItemId(int position) {
        return position;
    }

    /**
     * 生成每一项搜索结果的显示视图
     */
    @Override
    public View getView(int position, View convertView, ViewGroup parent) {
        ViewHolder holder;

        if (convertView == null) {
            // 外层卡片式根布局
            LinearLayout root = new LinearLayout(MainActivity.this);
            root.setOrientation(LinearLayout.HORIZONTAL);
            root.setGravity(Gravity.CENTER_VERTICAL);
            root.setMinimumHeight(dp(62));
            root.setPadding(dp(12), dp(10), dp(12), dp(10));

            // 创建圆角浅色背景
            GradientDrawable bg = new GradientDrawable();
            bg.setColor(0xFFF8FAFC);
            bg.setCornerRadius(dp(14));
            bg.setStroke(dp(1), 0xFFE2E8F0);
            root.setBackground(bg);

            // 左侧文字区域
            LinearLayout textContainer = new LinearLayout(MainActivity.this);
            textContainer.setOrientation(LinearLayout.VERTICAL);
            LinearLayout.LayoutParams textParams =
                new LinearLayout.LayoutParams(0, ViewGroup.LayoutParams.WRAP_CONTENT, 1f);
            textContainer.setLayoutParams(textParams);

            // 标题文本
            TextView title = new TextView(MainActivity.this);
            title.setTextColor(0xFF0F172A);
            title.setTextSize(TypedValue.COMPLEX_UNIT_SP, 14);
            title.setTypeface(Typeface.DEFAULT_BOLD);
            title.setMaxLines(1);

            // 地址或说明文本
            TextView snippet = new TextView(MainActivity.this);
            snippet.setTextColor(0xFF64748B);
            snippet.setTextSize(TypedValue.COMPLEX_UNIT_SP, 12);
            snippet.setMaxLines(2);
            LinearLayout.LayoutParams snippetLp =
                new LinearLayout.LayoutParams(ViewGroup.LayoutParams.WRAP_CONTENT,
                    ViewGroup.LayoutParams.WRAP_CONTENT);
            snippetLp.topMargin = dp(4);
            snippet.setLayoutParams(snippetLp);

            textContainer.addView(title);
            textContainer.addView(snippet);

            // 右侧箭头提示

```

```

        TextView arrow = new TextView(MainActivity.this);
        arrow.setText(">");
        arrow.setTextColor(0xFF94A3B8);
        arrow.setTextSize(TypedValue.COMPLEX_UNIT_SP, 18);

        // 把左右内容加到根布局中
        root.addView(textContainer);
        root.addView(arrow);

        // 使用 ViewHolder 缓存控件
        holder = new ViewHolder();
        holder.title = title;
        holder.snippet = snippet;
        holder.arrow = arrow;

        convertView = root;
        convertView.setTag(holder);
    } else {
        // 复用旧视图, 提升列表滚动性能
        holder = (ViewHolder) convertView.getTag();
    }

    PoiItem item = searchPoiList.get(position);

    // 如果标题为空, 则显示默认值
    String title = item.getTitle() == null ? "未知地点" : item.getTitle();
    // 如果地址为空, 则显示默认说明
    String snippet = item.getSnippet() == null || item.getSnippet().trim().isEmpty()
        ? "暂无详细地址"
        : item.getSnippet();

    holder.title.setText(title);
    holder.snippet.setText(snippet);

    return convertView;
}

/**
 * ViewHolder 用于缓存每一项中的子控件引用
 */
class ViewHolder {
    TextView title;
    TextView snippet;
    TextView arrow;
}
}

```

3.2 activity_main.xml

以下为 activity_main.xml 的完整注释版源码：

```
<?xml version="1.0" encoding="utf-8"?>
<!--
    GPSMap 主界面布局文件（详细注释版）
    =====
    文件作用：
    1. 使用 FrameLayout 作为根布局，让 MapView 全屏铺底，再将顶部信息卡片、功能模块、围栏半径卡片、底部操作栏和回到当前位置按钮叠加在地图上。
    2. 顶部卡片包含搜索栏、地址/坐标状态、三个核心摘要卡片和可展开的“功能模块”。
    3. “功能模块”默认收起，点击展开后显示卫星质量、精度评价、坐标转换、误差源、异常检测和组合导航六个标签按钮。
    4. 底部功能区保留围栏、拍照、轨迹及其对应清除按钮，保证原 App 基础功能仍然直观可用。
    5. 右侧回到当前位置按钮略微放大并下移，避免与顶部功能面板视觉冲突。

    修改原则：
    - 只通过布局层次、控件尺寸、显示/隐藏状态和 Material 风格卡片优化展示，不改变 Java 中的业务逻辑。
    - 各 TextView、Button 的 id 必须与 MainActivity.java 中 findViewById() 一一对应，否则会导致运行时空指针错误。
-->
<!--
    主界面布局文件
    整体采用 FrameLayout 作为根布局
    这样可以使地图作为底层全屏显示
    再把顶部卡片、底部功能区、侧边按钮叠加到地图之上
-->
<!--
    布局层次说明
    1. 最底层是高德地图 MapView
    2. 顶部是搜索与状态信息卡片
    3. 中下部是围栏半径调节卡片
    4. 底部是两行功能按钮区
    5. 右侧中间是回到当前位置按钮
-->
<FrameLayout xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    android:layout_width="match_parent"
    android:layout_height="match_parent">

    <!-- 地图主体区域，铺满整个屏幕，作为所有上层控件的背景 -->
    <com.amap.api.maps.MapView
        android:id="@+id/mapView"
        android:layout_width="match_parent"
        android:layout_height="match_parent" >

        <com.google.android.material.button.MaterialButton
            android:id="@+id/btnCenterMe"
            android:layout_width="56dp"
            android:layout_height="56dp"
            android:layout_gravity="end|center_vertical"
            android:layout_marginEnd="16dp"
            android:minWidth="0dp"
            android:minHeight="0dp"
            android:text="*"
            android:textAllCaps="false"
            android:textColor="#FFFFFF"
            android:textSize="20sp"
            android:translationY="170dp"
            app:backgroundTint="#475569"
            app:cornerRadius="28dp" />
    </com.amap.api.maps.MapView>

    <!-- 顶部搜索与状态卡片 -->
    <!-- 该卡片用于显示搜索框、搜索结果、路线操作按钮以及定位状态信息 -->
    <com.google.android.material.card.MaterialCardView
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:layout_marginStart="12dp"
        android:layout_marginTop="12dp"
        android:layout_marginEnd="12dp"
        app:cardBackgroundColor="#F7FFFFFF"
        app:cardCornerRadius="18dp"
        app:cardElevation="5dp"
        app:strokeColor="#E2E8F0"
        app:strokeWidth="1dp">

        <!-- 顶部卡片内部采用纵向线性布局 -->
        <!-- 里面依次放标题栏、搜索栏、搜索扩展区 -->
        <LinearLayout
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:orientation="vertical"
            android:padding="12dp">

            <!-- 顶部标题栏 -->
            <!-- 左侧显示应用标题，右侧显示展开收起按钮 -->
            <LinearLayout
                android:layout_width="match_parent"
                android:layout_height="wrap_content"
                android:gravity="center_vertical"
                android:orientation="horizontal">

                <!-- 页面标题 -->
                <!-- 宽度设为 0dp 并配合 weight=1，用来自动占满剩余空间 -->
                <TextView
```

```

        android:layout_width="0dp"
        android:layout_height="wrap_content"
        android:layout_weight="1"
        android:text="GPSmap"
        android:textColor="#1F2937"
        android:textSize="20sp"
        android:textStyle="bold" />

<!-- 展开收起按钮 -->
<!-- 用于控制下面状态区和搜索结果区是否显示 -->
<com.google.android.material.button.MaterialButton
    android:id="@+id/btnToggleInfo"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:minHeight="0dp"
    android:minWidth="0dp"
    android:paddingStart="12dp"
    android:paddingTop="6dp"
    android:paddingEnd="12dp"
    android:paddingBottom="6dp"
    android:text="收起"
    android:textAllCaps="false"
    android:textColor="FFFFFF"
    android:textSize="12sp"
    app:backgroundTint="#64748B"
    app:cornerRadius="14dp" />
</LinearLayout>

<!-- 搜索栏区域 -->
<!-- 左边是输入框，右边是搜索按钮 -->
<LinearLayout
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:layout_marginTop="10dp"
    android:orientation="horizontal">

    <!-- 搜索输入框外层卡片 -->
    <!-- 使用卡片包裹输入框，界面更圆润统一 -->
    <com.google.android.material.card.MaterialCardView
        android:layout_width="0dp"
        android:layout_height="40dp"
        android:layout_weight="1"
        app:cardBackgroundColor="#F8FAFC"
        app:cardCornerRadius="14dp"
        app:cardElevation="0dp"
        app:strokeColor="CBD5E1"
        app:strokeWidth="1dp">

        <!-- 输入框内部横向布局 -->
        <!-- 左边小图标，右边 EditText -->
        <LinearLayout
            android:layout_width="match_parent"
            android:layout_height="match_parent"
            android:gravity="center_vertical"
            android:orientation="horizontal"
            android:paddingStart="10dp"
            android:paddingEnd="10dp">

            <!-- 搜索图标 -->
            <!-- 这里直接使用字符模拟图标 -->
            <TextView
                android:layout_width="wrap_content"
                android:layout_height="wrap_content"
                android:text="🔍"
                android:textColor="#64748B"
                android:textSize="14sp" />

            <!-- 搜索输入框 -->
            <!-- 用户在这里输入要查询的地点关键字 -->
            <EditText
                android:id="@+id/etKeyword"
                android:layout_width="0dp"
                android:layout_height="match_parent"
                android:layout_marginStart="6dp"
                android:layout_weight="1"
                android:background="@android:color/transparent"
                android:hint="搜索位置"
                android:textColor="#0F172A"
                android:textColorHint="#94A3B8"
                android:textSize="13sp" />

        </LinearLayout>
    </com.google.android.material.card.MaterialCardView>

    <!-- 搜索按钮 -->
    <!-- 点击后触发 POI 搜索 -->
    <com.google.android.material.button.MaterialButton
        android:id="@+id/btnSearch"
        android:layout_width="wrap_content"
        android:layout_height="40dp"
        android:layout_marginStart="8dp"
        android:minWidth="0dp"
        android:text="搜索"
        android:textAllCaps="false"
        android:textColor="FFFFFF"
        android:textSize="12sp"
        app:backgroundTint="#475569"
        app:cornerRadius="14dp" />

```

```

</LinearLayout>

<!-- 搜索扩展区域 -->
<!-- 这一块可以整体展开和收起 -->
<!-- 包含搜索结果列表、路线按钮和状态文本 -->
<LinearLayout
    android:id="@+id/layoutSearchExtras"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:orientation="vertical">

    <!-- 搜索结果列表 -->
    <!-- 搜索到地点后在这里显示多个 POI 结果 -->
    <!-- 默认隐藏，只有搜索到结果时才显示 -->
    <ListView
        android:id="@+id/lvSearchResults"
        android:layout_width="match_parent"
        android:layout_height="190dp"
        android:layout_marginTop="8dp"
        android:divider="@android:color/transparent"
        android:dividerHeight="6dp"
        android:padding="2dp"
        android:clipToPadding="false"
        android:visibility="gone" />

    <!-- 路线操作区 -->
    <!-- 只有用户选中目的地之后才显示 -->
    <LinearLayout
        android:id="@+id/layoutRouteActions"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:layout_marginTop="8dp"
        android:orientation="vertical"
        android:visibility="gone">

        <!-- 当前已选目的地文字提示 -->
        <TextView
            android:id="@+id/tvSelectedDestination"
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:text="已选目的地: --"
            android:textColor="#0F172A"
            android:textSize="12sp"
            android:textStyle="bold" />

    <!-- 路线操作按钮行 -->
    <!-- 包含步行、驾车、清除路线三个按钮 -->
    <LinearLayout
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:layout_marginTop="6dp"
        android:orientation="horizontal">

        <!-- 步行路线按钮 -->
        <com.google.android.material.button.MaterialButton
            android:id="@+id/btnWalkRoute"
            android:layout_width="0dp"
            android:layout_height="34dp"
            android:layout_marginEnd="6dp"
            android:layout_weight="1"
            android:minWidth="0dp"
            android:text="步行"
            android:textAllCaps="false"
            android:textColor="FFFFFF"
            android:textSize="11sp"
            app:backgroundTint="#64748B"
            app:cornerRadius="14dp" />

        <!-- 驾车路线按钮 -->
        <com.google.android.material.button.MaterialButton
            android:id="@+id/btnDriveRoute"
            android:layout_width="0dp"
            android:layout_height="34dp"
            android:layout_marginEnd="6dp"
            android:layout_weight="1"
            android:minWidth="0dp"
            android:text="驾车"
            android:textAllCaps="false"
            android:textColor="FFFFFF"
            android:textSize="11sp"
            app:backgroundTint="#78716C"
            app:cornerRadius="14dp" />

        <!-- 清除路线按钮 -->
        <com.google.android.material.button.MaterialButton
            android:id="@+id/btnClearRoute"
            android:layout_width="0dp"
            android:layout_height="34dp"
            android:layout_weight="1"
            android:minWidth="0dp"
            android:text="清路"
            android:textAllCaps="false"
            android:textColor="FFFFFF"
            android:textSize="11sp"
            app:backgroundTint="#94A3B8"
            app:cornerRadius="14dp" />

    </LinearLayout>

```

```

</LinearLayout>

<!-- 状态文本区域 -->
<!-- 用于显示当前地址、经纬度、围栏状态、轨迹状态、路线状态 -->

<!-- 地址显示 -->
<TextView
    android:id="@+id/tvAddress"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:layout_marginTop="8dp"
    android:maxLines="2"
    android:text="地址：定位中..."
    android:textColor="#475569"
    android:textSize="12sp" />

<!-- 经纬度与精度显示 -->
<TextView
    android:id="@+id/tvCoord"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:layout_marginTop="3dp"
    android:text="经纬度：--"
    android:textColor="#475569"
    android:textSize="12sp" />

<!-- 功能模块摘要：默认只显示核心状态，具体模块可按需展开 -->
<LinearLayout
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:layout_marginTop="8dp"
    android:orientation="horizontal">

    <com.google.android.material.card.MaterialCardView
        android:layout_width="0dp"
        android:layout_height="58dp"
        android:layout_marginEnd="6dp"
        android:layout_weight="1"
        app:cardBackgroundColor="#F8FAFC"
        app:cardCornerRadius="14dp"
        app:cardElevation="0dp"
        app:strokeColor="#E2E8F0"
        app:strokeWidth="1dp">

        <LinearLayout
            android:layout_width="match_parent"
            android:layout_height="match_parent"
            android:gravity="center_vertical"
            android:orientation="vertical"
            android:padding="8dp">

            <TextView
                android:layout_width="wrap_content"
                android:layout_height="wrap_content"
                android:text="GNSS 质量"
                android:textColor="#64748B"
                android:textSize="10sp" />

            <TextView
                android:id="@+id/tvQuickGnss"
                android:layout_width="wrap_content"
                android:layout_height="wrap_content"
                android:layout_marginTop="2dp"
                android:text="等待卫星"
                android:textColor="#0F172A"
                android:textSize="13sp"
                android:textStyle="bold" />

        </LinearLayout>
    </com.google.android.material.card.MaterialCardView>

    <com.google.android.material.card.MaterialCardView
        android:layout_width="0dp"
        android:layout_height="58dp"
        android:layout_marginEnd="6dp"
        android:layout_weight="1"
        app:cardBackgroundColor="#F8FAFC"
        app:cardCornerRadius="14dp"
        app:cardElevation="0dp"
        app:strokeColor="#E2E8F0"
        app:strokeWidth="1dp">

        <LinearLayout
            android:layout_width="match_parent"
            android:layout_height="match_parent"
            android:gravity="center_vertical"
            android:orientation="vertical"
            android:padding="8dp">

            <TextView
                android:layout_width="wrap_content"
                android:layout_height="wrap_content"
                android:text="定位精度"
                android:textColor="#64748B"
                android:textSize="10sp" />

            <TextView
                android:id="@+id/tvQuickAccuracy"
                android:layout_width="wrap_content"
                android:layout_height="wrap_content"

```

```

        android:layout_marginTop="2dp"
        android:text="等待定位"
        android:textColor="#0F172A"
        android:textSize="13sp"
        android:textStyle="bold" />
    </LinearLayout>
</com.google.android.material.card.MaterialCardView>

<com.google.android.material.card.MaterialCardView
    android:layout_width="0dp"
    android:layout_height="58dp"
    android:layout_weight="1"
    app:cardBackgroundColor="#F8FAFC"
    app:cardCornerRadius="14dp"
    app:cardElevation="0dp"
    app:strokeColor="#E2E8F0"
    app:strokeWidth="1dp">

    <LinearLayout
        android:layout_width="match_parent"
        android:layout_height="match_parent"
        android:gravity="center_vertical"
        android:orientation="vertical"
        android:padding="8dp">

        <TextView
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:text="异常检测"
            android:textColor="#64748B"
            android:textSize="10sp" />

        <TextView
            android:id="@+id/tvQuickAnomaly"
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:layout_marginTop="2dp"
            android:text="待检测"
            android:textColor="#0F172A"
            android:textSize="13sp"
            android:textStyle="bold" />
    </LinearLayout>
</com.google.android.material.card.MaterialCardView>
</LinearLayout>

<!-- 功能模块标题行，默认收起，点击后显示具体功能按钮和详情卡片 -->
<LinearLayout
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:layout_marginTop="8dp"
    android:gravity="center_vertical"
    android:orientation="horizontal">

    <TextView
        android:layout_width="0dp"
        android:layout_height="wrap_content"
        android:layout_weight="1"
        android:text="功能模块"
        android:textColor="#334155"
        android:textSize="12sp"
        android:textStyle="bold" />

    <com.google.android.material.button.MaterialButton
        android:id="@+id/btnToggleFeatureModule"
        android:layout_width="wrap_content"
        android:layout_height="40dp"
        android:minWidth="0dp"
        android:minHeight="0dp"
        android:paddingStart="12dp"
        android:paddingEnd="12dp"
        android:text="展开"
        android:textAllCaps="false"
        android:textColor="FFFFFF"
        android:textSize="11sp"
        app:backgroundTint="#64748B"
        app:cornerRadius="15dp" />
</LinearLayout>

<LinearLayout
    android:id="@+id/layoutFeatureModuleContent"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:orientation="vertical"
    android:visibility="gone">

    <HorizontalScrollView
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:layout_marginTop="6dp"
        android:overScrollMode="never"
        android:scrollbars="none">

        <LinearLayout
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:orientation="horizontal">

            <com.google.android.material.button.MaterialButton
                android:id="@+id/btnShowGnss"

```

```

        android:layout_width="wrap_content"
        android:layout_height="32dp"
        android:layout_marginEnd="6dp"
        android:minWidth="0dp"
        android:paddingStart="12dp"
        android:paddingEnd="12dp"
        android:text="卫星质量"
        android:textAllCaps="false"
        android:textColor="#FFFFFF"
        android:textSize="11sp"
        app:backgroundTint="#475569"
        app:cornerRadius="16dp" />

<com.google.android.material.button.MaterialButton
    android:id="@+id/btnShowAccuracy"
    android:layout_width="wrap_content"
    android:layout_height="32dp"
    android:layout_marginEnd="6dp"
    android:minWidth="0dp"
    android:paddingStart="12dp"
    android:paddingEnd="12dp"
    android:text="精度评价"
    android:textAllCaps="false"
    android:textColor="#334155"
    android:textSize="11sp"
    app:backgroundTint="#E2E8F0"
    app:cornerRadius="16dp" />

<com.google.android.material.button.MaterialButton
    android:id="@+id/btnShowCoord"
    android:layout_width="wrap_content"
    android:layout_height="32dp"
    android:layout_marginEnd="6dp"
    android:minWidth="0dp"
    android:paddingStart="12dp"
    android:paddingEnd="12dp"
    android:text="坐标转换"
    android:textAllCaps="false"
    android:textColor="#334155"
    android:textSize="11sp"
    app:backgroundTint="#E2E8F0"
    app:cornerRadius="16dp" />

<com.google.android.material.button.MaterialButton
    android:id="@+id/btnShowError"
    android:layout_width="wrap_content"
    android:layout_height="32dp"
    android:layout_marginEnd="6dp"
    android:minWidth="0dp"
    android:paddingStart="12dp"
    android:paddingEnd="12dp"
    android:text="误差源"
    android:textAllCaps="false"
    android:textColor="#334155"
    android:textSize="11sp"
    app:backgroundTint="#E2E8F0"
    app:cornerRadius="16dp" />

<com.google.android.material.button.MaterialButton
    android:id="@+id/btnShowAnomaly"
    android:layout_width="wrap_content"
    android:layout_height="32dp"
    android:layout_marginEnd="6dp"
    android:minWidth="0dp"
    android:paddingStart="12dp"
    android:paddingEnd="12dp"
    android:text="异常检测"
    android:textAllCaps="false"
    android:textColor="#334155"
    android:textSize="11sp"
    app:backgroundTint="#E2E8F0"
    app:cornerRadius="16dp" />

<com.google.android.material.button.MaterialButton
    android:id="@+id/btnShowSensor"
    android:layout_width="wrap_content"
    android:layout_height="32dp"
    android:minWidth="0dp"
    android:paddingStart="12dp"
    android:paddingEnd="12dp"
    android:text="组合导航"
    android:textAllCaps="false"
    android:textColor="#334155"
    android:textSize="11sp"
    app:backgroundTint="#E2E8F0"
    app:cornerRadius="16dp" />
</LinearLayout>
</HorizontalScrollView>

<com.google.android.material.card.MaterialCardView
    android:id="@+id/cardFeatureDetail"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:layout_marginTop="8dp"
    app:cardBackgroundColor="#F8FAFC"
    app:cardCornerRadius="16dp"
    app:cardElevation="0dp"
    app:strokeColor="#CBD5E1"

```

```

        app:strokeWidth="1dp">

        <LinearLayout
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:orientation="vertical"
            android:padding="10dp">

            <TextView
                android:id="@+id/tvFeatureTitle"
                android:layout_width="wrap_content"
                android:layout_height="wrap_content"
                android:text="卫星质量评估"
                android:textColor="#0F172A"
                android:textSize="13sp"
                android:textStyle="bold" />

            <TextView
                android:id="@+id/tvFeatureContent"
                android:layout_width="match_parent"
                android:layout_height="wrap_content"
                android:layout_marginTop="5dp"
                android:lineSpacingExtra="2dp"
                android:maxLines="4"
                android:text="点击上方功能标签查看详细评估结果"
                android:textColor="#475569"
                android:textSize="12sp" />

            </LinearLayout>
        </com.google.android.material.card.MaterialCardView>
    </LinearLayout>

    <!-- 以下 TextView 用于保存各项功能的完整文字，界面通过上方详情卡片按需显示 -->
    <TextView
        android:id="@+id/tvCoordConverted"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="坐标转换: --"
        android:visibility="gone" />

    <TextView
        android:id="@+id/tvGnssQuality"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="GNSS 质量: 等待卫星数据"
        android:visibility="gone" />

    <TextView
        android:id="@+id/tvAccuracyEval"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="定位精度评价: 等待定位数据"
        android:visibility="gone" />

    <TextView
        android:id="@+id/tvErrorAnalysis"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="误差源分析: 等待定位数据"
        android:visibility="gone" />

    <TextView
        android:id="@+id/tvAnomalyDetect"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="异常/欺骗初筛: 待检测"
        android:visibility="gone" />

    <TextView
        android:id="@+id/tvSensorFusion"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="组合导航: 等待传感器数据"
        android:visibility="gone" />

    <!-- 围栏状态显示 -->
    <TextView
        android:id="@+id/tvFenceStatus"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_marginTop="3dp"
        android:text="围栏: 未设置"
        android:textColor="#64748B"
        android:textSize="12sp"
        android:textStyle="bold" />

    <!-- 轨迹状态显示 -->
    <TextView
        android:id="@+id/tvTrackStatus"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_marginTop="3dp"
        android:text="轨迹: 未记录"
        android:textColor="#64748B"
        android:textSize="12sp"
        android:textStyle="bold" />

    <!-- 路线状态显示 -->
    <TextView

```

```

        android:id="@+id/tvRouteStatus"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_marginTop="3dp"
        android:text="路线：未规划"
        android:textColor="#64748B"
        android:textSize="12sp"
        android:textStyle="bold" />
    </LinearLayout>
</LinearLayout>
</com.google.android.material.card.MaterialCardView>

<!-- 围栏半径调节卡片 -->
<!-- 当开启围栏模式后显示，用于调节电子围栏半径 -->
<!-- 默认隐藏 -->
<com.google.android.material.card.MaterialCardView
    android:id="@+id/cardFenceRadius"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:layout_gravity="bottom"
    android:layout_marginStart="12dp"
    android:layout_marginEnd="12dp"
    android:layout_marginBottom="86dp"
    android:visibility="gone"
    app:cardBackgroundColor="#F7FFFFFF"
    app:cardCornerRadius="16dp"
    app:cardElevation="4dp"
    app:strokeColor="#E2E8F0"
    app:strokeWidth="1dp">

    <!-- 围栏半径卡片内部布局 -->
    <LinearLayout
        android:id="@+id/layoutFenceRadius"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:orientation="vertical"
        android:padding="10dp">

        <!-- 当前围栏半径文字 -->
        <TextView
            android:id="@+id/tvFenceRadius"
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:text="围栏半径：100m"
            android:textColor="#334155"
            android:textSize="12sp"
            android:textStyle="bold" />

        <!-- 围栏半径调节条 -->
        <!-- 用户拖动后可改变电子围栏的大小 -->
        <SeekBar
            android:id="@+id/seekFenceRadius"
            android:layout_width="match_parent"
            android:layout_height="wrap_content" />
    </LinearLayout>
</com.google.android.material.card.MaterialCardView>

<!-- 底部功能区卡片 -->
<!-- 包含围栏、拍照、轨迹及其对应的清除按钮 -->
<com.google.android.material.card.MaterialCardView
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:layout_gravity="bottom"
    android:layout_marginStart="12dp"
    android:layout_marginEnd="12dp"
    android:layout_marginBottom="18dp"
    app:cardBackgroundColor="#F7FFFFFF"
    app:cardCornerRadius="18dp"
    app:cardElevation="5dp"
    app:strokeColor="#E2E8F0"
    app:strokeWidth="1dp">

    <!-- 底部卡片内部纵向布局 -->
    <!-- 分成上下两行按钮 -->
    <LinearLayout
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:orientation="vertical"
        android:padding="8dp">

        <!-- 第一行按钮 -->
        <!-- 围栏、拍照、轨迹 -->
        <LinearLayout
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:orientation="horizontal">

            <!-- 开启或关闭围栏模式按钮 -->
            <com.google.android.material.button.MaterialButton
                android:id="@+id/btnFenceMode"
                android:layout_width="0dp"
                android:layout_height="34dp"
                android:layout_marginEnd="6dp"
                android:layout_weight="1"
                android:minWidth="0dp"
                android:text="围栏"
                android:textAllCaps="false"

```

```

        android:textColor="#334155"
        android:textSize="10sp"
        app:backgroundTint="#E2E8F0"
        app:cornerRadius="14dp" />

<!-- 拍照定位按钮 -->
<com.google.android.material.button.MaterialButton
    android:id="@+id/btnTakePhoto"
    android:layout_width="0dp"
    android:layout_height="34dp"
    android:layout_marginEnd="6dp"
    android:layout_weight="1"
    android:minWidth="0dp"
    android:text="拍照"
    android:textAllCaps="false"
    android:textColor="#334155"
    android:textSize="10sp"
    app:backgroundTint="#E2E8F0"
    app:cornerRadius="14dp" />

<!-- 轨迹记录开始或停止按钮 -->
<com.google.android.material.button.MaterialButton
    android:id="@+id/btnTrack"
    android:layout_width="0dp"
    android:layout_height="34dp"
    android:layout_weight="1"
    android:minWidth="0dp"
    android:text="轨迹"
    android:textAllCaps="false"
    android:textColor="#334155"
    android:textSize="10sp"
    app:backgroundTint="#E2E8F0"
    app:cornerRadius="14dp" />
</LinearLayout>

<!-- 第二行按钮 -->
<!-- 清围、清拍、清轨 -->
<LinearLayout
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:layout_marginTop="6dp"
    android:orientation="horizontal">

    <!-- 清除电子围栏按钮 -->
    <com.google.android.material.button.MaterialButton
        android:id="@+id/btnClearFence"
        android:layout_width="0dp"
        android:layout_height="34dp"
        android:layout_marginEnd="6dp"
        android:layout_weight="1"
        android:minWidth="0dp"
        android:text="清围"
        android:textAllCaps="false"
        android:textColor="#334155"
        android:textSize="10sp"
        app:backgroundTint="#E2E8F0"
        app:cornerRadius="14dp" />

    <!-- 清除拍照点按钮 -->
    <com.google.android.material.button.MaterialButton
        android:id="@+id/btnClearPhotos"
        android:layout_width="0dp"
        android:layout_height="34dp"
        android:layout_marginEnd="6dp"
        android:layout_weight="1"
        android:minWidth="0dp"
        android:text="清拍"
        android:textAllCaps="false"
        android:textColor="#334155"
        android:textSize="10sp"
        app:backgroundTint="#E2E8F0"
        app:cornerRadius="14dp" />

    <!-- 清除轨迹按钮 -->
    <com.google.android.material.button.MaterialButton
        android:id="@+id/btnClearTrack"
        android:layout_width="0dp"
        android:layout_height="34dp"
        android:layout_weight="1"
        android:minWidth="0dp"
        android:text="清轨"
        android:textAllCaps="false"
        android:textColor="#334155"
        android:textSize="10sp"
        app:backgroundTint="#E2E8F0"
        app:cornerRadius="14dp" />
    </LinearLayout>
</LinearLayout>
</com.google.android.material.card.MaterialCardView>

<!-- 侧边回到当前位置按钮 -->
<!-- 点击后地图镜头重新移动到当前定位点 -->

</FrameLayout>

```

3.3 AndroidManifest.xml

以下为 AndroidManifest.xml 的完整注释版源码：

```
<?xml version="1.0" encoding="utf-8"?>
<!--
    GPSMap 应用清单文件（详细注释版）
    =====
    文件作用：
    1. 声明 App 所需权限，例如网络、定位、WiFi 状态、保持唤醒和相机权限。
    2. 声明 GPS/GNSS 与相机硬件特性，并通过 required=false 保证缺少部分硬件的设备仍可安装。
    3. 配置高德地图 SDK 的 API Key，使地图、定位、搜索和路线服务能够正常使用。
    4. 声明 MainActivity 为应用入口，保证用户点击桌面图标后进入主地图界面。

    注意事项：
    - ACCESS_FINE_LOCATION 是读取高精度定位和 GNSS 卫星状态的关键权限。
    - CAMERA 权限用于拍照定位；如果不使用拍照功能，可在功能裁剪时删除该权限和对应代码。
    - API Key 应与包名、SHA1 等高德开放平台配置保持一致，否则地图服务可能无法加载。
-->
<!-- Android 应用清单文件，负责声明权限、硬件特性、应用信息和入口 Activity -->
<manifest xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:tools="http://schemas.android.com/tools">

    <!-- 允许应用访问网络，地图加载、POI 搜索、路径规划等都需要联网 -->
    <uses-permission android:name="android.permission.INTERNET" />

    <!-- 允许应用获取当前网络连接状态 -->
    <uses-permission android:name="android.permission.ACCESS_NETWORK_STATE" />

    <!-- 允许应用获取 WiFi 状态信息 -->
    <uses-permission android:name="android.permission.ACCESS_WIFI_STATE" />

    <!-- 允许应用获取粗略位置信息 -->
    <uses-permission android:name="android.permission.ACCESS_COARSE_LOCATION" />

    <!-- 允许应用获取精确位置信息，这是定位导航功能的核心权限 -->
    <uses-permission android:name="android.permission.ACCESS_FINE_LOCATION" />

    <!-- 允许应用修改 WiFi 状态 -->
    <uses-permission android:name="android.permission.CHANGE_WIFI_STATE" />

    <!-- 允许应用保持设备唤醒，避免定位过程中设备休眠影响功能 -->
    <uses-permission android:name="android.permission.WAKE_LOCK" />

    <!-- 允许应用调用相机，用于拍照定位功能 -->
    <uses-permission android:name="android.permission.CAMERA" />

    <!-- 声明应用可使用 GNSS/GPS 硬件 -->
    <!-- required=false 表示没有独立 GNSS 芯片时应用依然可以安装，只是定位质量评估信息可能不完整 -->
    <uses-feature
        android:name="android.hardware.location.gps"
        android:required="false" />

    <!-- 声明应用可使用摄像头硬件 -->
    <!-- required=false 表示没有摄像头时应用依然可以安装，只是相关功能不可用 -->
    <uses-feature
        android:name="android.hardware.camera"
        android:required="false" />

    <!-- application 节点定义整个应用的全局属性 -->
    <!-- allowBackup 表示允许系统备份应用数据 -->
    <!-- dataExtractionRules 和 fullBackupContent 是备份相关配置文件 -->
    <!-- icon 和 roundIcon 分别是普通图标和圆形图标 -->
    <!-- label 是应用名称 -->
    <!-- supportsRtl 表示支持从右到左布局 -->
    <!-- theme 指定应用主题 -->
    <!-- tools:targetApi 表示这里的配置面向的目标 API 版本 -->
    <application
        android:allowBackup="true"
        android:dataExtractionRules="@xml/data_extraction_rules"
        android:fullBackupContent="@xml/backup_rules"
        android:icon="@mipmap/ic_launcher"
        android:label="@string/app_name"
        android:roundIcon="@mipmap/ic_launcher_round"
        android:supportsRtl="true"
        android:theme="@style/Theme.GPSMap"
        tools:targetApi="31">

        <!-- 配置高德地图 SDK 的 API Key -->
        <meta-data
            android:name="com.amap.api.v2.apikey"
            android:value="86c47ce5ealed02ff40457e316d0c6a8" />

        <!-- 声明主界面 Activity -->
        <!-- exported=true 表示该 Activity 可被系统启动，因为它是桌面入口 -->
        <activity
            android:name=".MainActivity"
            android:exported="true">

            <!-- intent-filter 表示该 Activity 的启动方式 -->
            <!-- MAIN 表示主入口动作 -->
            <!-- LAUNCHER 表示会出现在桌面启动器中 -->
```

```
        <intent-filter>
            <action android:name="android.intent.action.MAIN" />
            <category android:name="android.intent.category.LAUNCHER" />
        </intent-filter>
    </activity>

</application>

</manifest>
```

3.4 build.gradle

以下为 build.gradle 的完整注释版源码：

```
/*
 * GPSmap 模块构建配置文件（详细注释版）
 * =====
 * 文件作用：
 * 1. 声明当前模块为 Android Application，可被编译、安装并运行在 Android 设备上。
 * 2. 配置 namespace、applicationId、minSdk、targetSdk、versionCode、versionName 等基础构建信息。
 * 3. 限定 NDK ABI 架构，减少不必要的 so 库体积。
 * 4. 引入 AppCompat、Material、JUnit、Espresso 以及高德 3D 地图/定位/搜索一体化 SDK。
 *
 * 注意事项：
 * - 高德依赖使用 latest.integration，便于课程开发阶段快速接入，但正式项目更建议固定具体版本号以避免依赖自动升级带来的兼容性问题。
 * - 当前 compileSdk/targetSdk 为 36，如本地 Android Studio 没有安装相应 SDK，需要在 SDK Manager 中安装或根据本机环境调整。
 */
plugins {
    // 通过版本目录（Version Catalog）里的别名引入 Android Application 插件。
    // 这说明当前模块是一个“可安装的 Android 应用模块”，而不是普通的 Java 库或 Android Library。
    alias(libs.plugins.android.application)
}

// Android 模块的核心构建配置都写在 android { } 代码块中。
android {
    // 设置当前应用的命名空间（namespace）。
    // 它通常对应生成的 R 类、BuildConfig 类所在的包名空间。
    // 这里使用 com.example.gpsmap，和你的应用包结构保持一致。
    namespace 'com.example.gpsmap'

    // 配置编译 SDK 版本。
    // compileSdk 决定项目在编译时使用哪一版 Android SDK API。
    // 编译 SDK 越高，越可以使用新的系统 API。
    compileSdk {
        // 指定主 SDK 版本号为 36。
        // release(36) 表示使用 Android API 36 的正式发布版本配置。
        version = release(36) {
            // 指定 minorApiLevel 为 1。
            // 这是更细粒度的 API 版本控制，用于一些新式 SDK/预览发布配置场景。
            minorApiLevel = 1
        }
    }

    // defaultConfig 是默认构建配置。
    // 这里的配置会作用于所有构建变体（如 debug、release），除非被其他构建类型单独覆盖。
    defaultConfig {
        // 设置应用的唯一包名标识（Application ID）。
        // 应用安装到手机后，系统实际识别的就是这个 ID。
        applicationId "com.example.gpsmap"

        // 指定应用支持的最低 Android 系统版本。
        // minSdk 24 表示 Android 7.0 (Nougat) 及以上设备才能安装。
        minSdk 24

        // 指定应用面向适配的目标 Android 版本。
        // targetSdk 越高，表示应用越按新系统行为规范运行。
        targetSdk 36

        // 设置应用内部版本号。
        // versionCode 必须是整数，主要给系统和应用商店用于版本升级判断。
        versionCode 1

        // 设置应用对用户可见的版本名。
        // 这个字符串通常会显示在应用详情页中。
        versionName "1.0"

        // 指定仪器化测试（Instrumentation Test）运行器。
        // 当你运行 androidTest 目录下的测试代码时，会由这个 Runner 启动测试环境。
        testInstrumentationRunner "androidx.test.runner.AndroidJUnitRunner"

        // 配置 NDK (Native Development Kit) 相关参数。
        // 如果项目依赖到带有 native so 库的第三方 SDK，这里常常需要限制 ABI。
        ndk {
            // 指定仅打包这两种 CPU 架构的 so 库：
            // armeabi-v7a: 32 位 ARM
            // arm64-v8a: 64 位 ARM
            // 这样可以减少 APK/AAB 体积，也避免打包不需要的架构。
            abiFilters 'armeabi-v7a', 'arm64-v8a'
        }
    }

    // 配置不同的构建类型（Build Types）。
    // 常见的有 debug 和 release。
    buildTypes {
        // release 表示正式发布版本的配置。
        release {
            // 是否开启代码压缩、混淆、优化。
            // false 表示当前不启用混淆，也不会删除未使用代码。
            // 对课程作业或调试阶段比较方便，但正式发布一般会考虑开启。
            minifyEnabled false
        }
    }
}
```

```

        // 指定混淆规则文件。
        // getDefaultProguardFile(...): 使用 Android 默认提供的优化混淆规则模板。
        // 'proguard-rules.pro': 项目自定义混淆规则文件。
        // 虽然当前 minifyEnabled 为 false, 这两项暂时不会生效, 但保留配置是常见做法。
        proguardFiles getDefaultProguardFile('proguard-android-optimize.txt'), 'proguard-rules.pro'
    }
}

// 配置 Java 编译选项。
compileOptions {
    // 指定源代码使用的 Java 语言版本为 Java 11。
    // 这决定了你可以在代码中使用哪些 Java 语法特性。
    sourceCompatibility JavaVersion.VERSION_11

    // 指定编译后目标字节码版本也为 Java 11。
    // 通常应与 sourceCompatibility 保持一致。
    targetCompatibility JavaVersion.VERSION_11
}

// 声明项目依赖库。
dependencies {
    // 引入 AndroidX AppCompat 支持库。
    // 提供向下兼容的 Activity、主题、控件等基础能力。
    implementation libs.appcompat

    // 引入 Material Design 组件库。
    // 你的 MaterialButton、MaterialCardView 等控件就依赖这个库。
    implementation libs.material

    // 引入本地单元测试框架 JUnit。
    // 用于 test 目录下的 JVM 单元测试。
    testImplementation libs.junit

    // 引入 AndroidX 扩展 JUnit 测试库。
    // 用于 androidTest 仪器化测试中的 JUnit 支持。
    androidTestImplementation libs.ext.junit

    // 引入 Espresso UI 测试框架核心库。
    // 可用于编写界面自动化测试, 例如点击按钮、检查文本显示等。
    androidTestImplementation libs.espresso.core

    // 引入高德地图 3D 地图 + 定位 + 搜索 一体化 SDK。
    // 这个依赖是你当前地图显示、定位、POI 搜索、路线规划等功能的核心基础。
    // latest.integration 表示始终使用该依赖的最新集成版本。
    // 优点是方便快速接入; 缺点是版本不可完全固定, 后期可能因远程更新带来兼容性变化。
    implementation 'com.amap.api:3dmap-location-search:latest.integration'
}

```

四、结果

GPSmap真机运行界面与功能展示总览



图 4-1 GPSmap 真机运行界面与功能展示总览

从总览图可以看出，GPSmap 采用地图作为全屏底图，上方为搜索和状态展示卡片，下方为围栏、拍照、轨迹等基础操作区，右侧设置回到当前位置的圆形按钮。功能模块既可以收起以减少遮挡，也可以展开后逐项查看 GNSS 扩展结果。整体界面比单纯堆叠文字更清晰，能兼顾日常使用和课堂汇报展示。

4.1 GNSS 定位质量评估界面



图 4-2 GNSS 定位质量评估界面

该界面展示了 GNSS 质量评估模块。顶部摘要卡片中显示“GNSS 质量：良 12/92 颗”，表示当前可见卫星数量较多，但实际参与定位的卫星为 12 颗，软件据此给出“良”的质量等级。

展开“卫星质量”标签后，下方详情卡片显示可见卫星、参与定位卫星、平均 C/N0、PDOP、HDOP、VDOP 以及星座统计。截图中平均 C/N0 约 33.4dB-Hz，HDOP 约 0.8，说明信号强度和水平几何条件较好。该功能把原本隐藏在系统底层的 GNSS 卫星状态转化为用户可读的质量评价。

4.2 GNSS 定位精度评价界面



图 4-3 GNSS 定位精度评价界面

该界面展示了定位精度评价模块。截图中系统水平精度为 59.2m，软件将其划分为“差”，并在详情中给出“建议移动到开阔区域”的提示。

这一结果说明本 App 不仅显示经纬度，还会结合精度阈值判断当前定位是否可靠。当定位精度较差时，用户可以根据提示判断可能处在遮挡较重、室内、楼间或信号不稳定区域，从而避免直接使用低质量位置进行导航或围栏判断。

4.3 坐标转换功能界面



图 4-4 坐标转换功能界面

该界面展示了坐标转换模块。上方定位坐标为 GCJ-02 坐标，这是高德地图常用坐标；详情卡片中同步给出 WGS84 和 BD-09 坐标。

通过该模块，用户可以直观看同一位置在不同坐标系下的数值差异。该功能适合说明移动地图开发中常见的坐标系问题，也方便后续与 GNSS 原始坐标、百度地图坐标或其他地理数据进行转换和对比。

4.4 导航误差源分析界面



图 4-5 导航误差源分析界面

该界面展示了导航误差源分析模块。截图中水平精度达到 4.0m，卫星质量良好，软件判断“卫星信号和几何条件较正常”。

误差源分析模块的作用不是简单给出一个数值，而是解释定位质量变化的可能原因。当卫星数不足、C/N0 偏低或 DOP 偏大时，软件会提示遮挡、多路径、信号弱或几何构型差等可能误差来源；当条件较好时，则给出正常判断。

4.5 欺骗与异常检测界面



图 4-6 欺骗与异常检测界面

该界面展示了异常检测模块。截图中软件判断为“正常”，并显示未发现明显跳变，最近跳变约 0.6m、推算速度约 0.3m/s。

异常检测主要用于发现定位结果是否存在突变或与运动状态矛盾的情况。若短时间内位置大幅跳变、推算速度异常，或卫星状态与系统精度不匹配，软件会给出轻微异常或疑似异常提示。该功能适合作为 GNSS 欺骗与异常检测的入门级展示。

4. 6 GNSS+惯性传感器组合导航界面



图 4-7 GNSS+惯性传感器组合导航界面

该界面展示了组合导航模块。截图中传感器航向约 165°，GNSS 速度为 0.0m/s，GNSS 航向为 0°，软件提示低速或静止状态下航向以手机传感器为参考。

这一设计体现了 GNSS 与惯性/磁传感器的互补关系：当手机处于静止或低速状态时，GNSS 航向容易受定位噪声影响；此时加速度计和磁力计估计出的手机姿态航向更适合作为方向参考。

4.7 电子围栏与轨迹记录界面



图 4-8 电子围栏与轨迹记录界面

该界面展示了功能模块收起后的主界面状态，同时包含电子围栏和轨迹记录结果。截图中围栏半径设置为 70m，系统显示当前位置在围栏外，距离中心约 116.4m；轨迹处于记录中，记录了 8 个点，总距离约 139.8m。

该界面说明基础导航功能仍然保留，并没有因为新增 GNSS 扩展功能而被削弱。功能模块收起后，上方卡片高度明显降低，用户可以更清楚地看到地图、围栏圆形范围、轨迹折线和当前位置箭头。

4.8 地点搜索与 POI 结果界面



图 4-9 地点搜索与 POI 结果界面

该界面展示了地点搜索功能。用户在搜索框中输入“东北大学”，系统返回多个 POI 候选结果，包括南湖校区、浑南校区、沈河校区等。

搜索结果采用卡片式列表展示，避免只返回一个模糊结果。用户可以根据名称和地址选择更准确的目的地。该功能为后续路线规划提供目标点，也使 App 具备更完整的导航应用体验。

4.9 路线规划结果界面

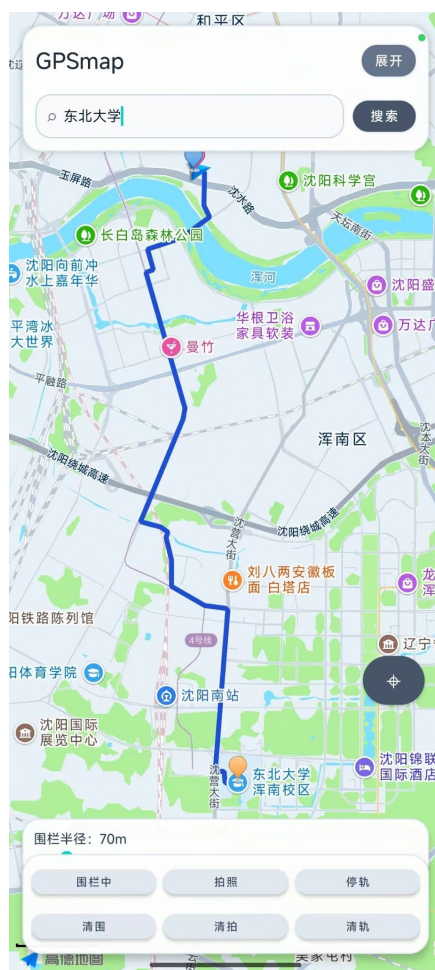


图 4-10 路线规划结果界面

该界面展示了路线规划结果。地图上绘制了从当前位置到目标地点的蓝色路线折线，并自动缩放地图，使完整路线能够在屏幕中显示。

路线规划功能与搜索结果联动：用户先搜索并选择目的地，再选择步行或驾车路线。该截图说明 App 已经具备基本导航 App 的核心流程，即定位当前位置、搜索目的地、生成路径并在地图上直观展示。

4.10 拍照定位信息界面

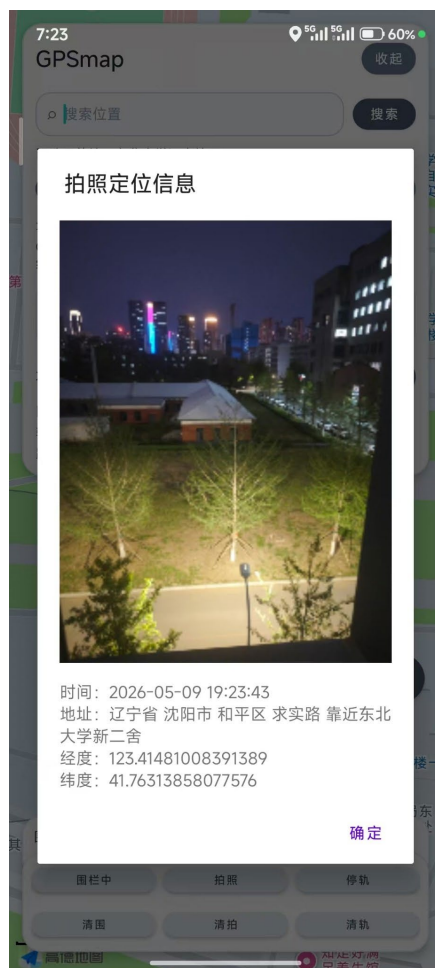


图 4-11 拍照定位信息界面

该界面展示了拍照定位结果。用户点击拍照后，App 调用系统相机，拍摄完成后在地图上生成拍照定位点；点击该点会弹出照片、时间、地址、经度和纬度信息。

拍照定位将影像信息与空间位置信息关联起来，适合用于外业记录、位置打卡、现场调查等场景。截图中展示了照片内容、拍摄时间、具体地址和坐标，说明该功能实现了“照片+位置+时间”的综合记录。

4.11 综合结果评价

从上述真机界面截图可以看出，本项目最终形成了较完整的导航定位 App 功能体系。基础层面，软件能够完成实时定位、地图显示、地点搜索、路线规划、电子围栏、轨迹记录和拍照定位；扩展层面，软件能够完成坐标转换、GNSS 定位质量评估、定位精度评价、误差源分析、异常检测和 GNSS+惯性传感器组合导航展示。

界面设计方面，软件最终没有采用大段文字堆叠的方式，而是通过“摘要卡片+功能模块展开”的结构突出重点。默认状态下只显示核心结论，减少对地图的遮挡；需要汇报或查看细节时，再展开功能模块逐项查看。这样的设计兼顾了美观性、实用性和课堂展示效果。

测试结果方面，截图中能够看到 GNSS 卫星数、C/N0、DOP、水平精度、坐标转换结果、异常检测结果和传感器航向等信息均能正常显示。不同截图中精度从 3.8m 到 59.2m 不等，也说明软件能够随定位环境变化更新状态，而不是固定显示某个静态结果。

五、总结

本次开发以 GPSmap 为基础，对原有 Android 导航定位 App 进行了较系统的功能扩展和界面优化。原软件已经具备地图显示、实时定位、搜索、路线、电子围栏、轨迹记录和拍照定位等基础能力；本次进一步围绕任务二要求，加入了坐标转换、GNSS 定位精度评价、GNSS 定位质量评估、GNSS 导航误差源分析、GNSS+惯性传感器组合导航、GNSS 欺骗与异常检测初筛等多个功能模块。虽然任务要求至少实现两项功能，但本项目最终完成了六项扩展功能，能够更全面地体现导航定位 App 的改进效果。

从功能实现角度看，坐标转换模块实现了 GCJ-02、WGS84 和 BD-09 坐标之间的联动显示，解决了地图开发中常见的坐标系统差异问题；GNSS 质量评估模块读取可见卫星数、参与定位卫星数、C/N0 和 DOP 等信息，使用户能够判断当前定位是否具有较好的卫星基础；定位精度评价模块将系统水平精度转化为“优、良、中、差”等直观等级；误差源分析模块根据卫星数量、信号强度和几何条件解释可能误差来源；异常检测模块通过跳变距离和推算速度判断定位结果是否存在异常；组合导航模块利用手机加速度计和磁力计估计航向，展示了 GNSS 与惯性传感器互补的思想。

通过本次开发，我进一步理解了 Android 移动端导航定位 App 的整体开发流程，包括高德地图 SDK 的初始化与隐私合规处理、定位权限申请、MapView 生命周期管理、POI 搜索、路线规划、Marker 和 Polyline 绘制、电子围栏半径判断、轨迹点记录、拍照回调以及动态界面更新等内容。同时，我也学习到 GNSS 质量并不只是一个经纬度结果，而是由卫星数量、信号强度、卫星空间几何、定位方式、速度、航向和环境遮挡等多种因素共同决定。

本项目也让我认识到手机导航定位与专业 GNSS 测量之间存在区别。手机能够读取一部分 GNSS 卫星状态和定位结果，但并不等同于专业接收机的原始观测解算。；而 CORS、RTK 或 PPP 等高精度定位技术还需要更完整的观测数据、差分改正数和严密的解算模型。

后续如果继续完善，可以从四个方向改进：一是加入 GNSS 质量日志保存功能，将卫星数、C/N0、DOP 和精度随时间记录下来，生成统计曲线；二是接入更专业的 RTK 或 CORS 改正数据，提高定位精度；三是加入轨迹误差分析功能，对比不同环境下的定位偏移；四是进一步引入滤波算法，例如卡尔曼滤波或互补滤波，将 GNSS 定位、速度、航向和惯性传感器数据进行更严密的融合。总体而言，本次任务不仅完成了软件功能扩展，也加深了我对移动导航定位、GNSS 质量评价和 Android 地图开发的理解。